



Grado en Ingeniería del Software

Detección de anomalías en robots industriales y activos móviles (AGV/AMR) mediante Machine Learning para Mantenimiento Predictivo.

Anomaly Detection in Industrial Mobile Assets (AGV/AMR) using Machine Learning for Predictive Maintenance

Realizado por

Rafael Sáez Arana

Tutorizado por

Cristian Martin Fernández

Co-tutorizado por

Bartolomé Rubio Muñoz

Departamento

Departamento de Lenguajes y Ciencias de la Computación

Málaga, November 12, 2026





ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

GRADO EN INGENIERÍA DEL SOFTWARE

Detección de anomalías en robots industriales y activos móviles (AGV/AMR) mediante Machine Learning para Mantenimiento Predictivo.

Anomaly Detection in Industrial Mobile Assets (AGV/AMR) using Machine Learning for Predictive Maintenance

Realizado por

Rafael Sáez Arana

Tutorizado por

Cristian Martin Fernández

Co-tutorizado por

Bartolomé Rubio Muñoz

Departamento

Departamento de Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA

MÁLAGA, NOVEMBER 12, 2026

Fecha defensa: septiembre de 2025

Resumen

En el marco de la transición hacia la Industria 4.0, los activos industriales y vehículos de guiado autónomo han evolucionado hacia sistemas capaces de emitir volúmenes continuos de telemetría en tiempo real, generando un gran caudal de información. Sin embargo, la simple recolección de estos datos no aporta valor operativo por sí sola. Únicamente mediante un procesamiento y análisis adecuados es posible transformar dichos datos en conocimiento útil para la toma de decisiones estratégicas. De ahí surge la idea de este proyecto, desarrollado en colaboración con EDAG, entidad de referencia en el sector industrial.

A pesar del avance en el mantenimiento predictivo durante los últimos años, existe una brecha notable entre los estudios teóricos sobre detección de anomalías y su aplicación efectiva en entornos de producción. Esta limitación se debe principalmente a la complejidad en la recolección de datos de calidad, al dinamismo de los entornos industriales y a la necesidad de procesar la información con bajas latencias para actuar antes de que ocurra una avería.

Para dar respuesta a esta problemática, este proyecto presenta una solución integral para la detección de anomalías en tiempo real sobre una flota de robots móviles. El sistema se compone de una infraestructura distribuida de ingesta y procesamiento continuo de datos, una capa de almacenamiento histórico optimizada para series temporales y un módulo analítico basado en aprendizaje automático para la identificación de comportamientos anómalos. Todo el conjunto se completa con una herramienta visual de monitorización y alertas para el personal de planta, desarrollada bajo una arquitectura limpia, desacoplada y escalable.

Palabras clave: IIoT, Mantenimiento Predictivo, Detección de Anomalías, Aprendizaje Automático, Robots Móviles, Industria 4.0, Data Lakehouse, Arquitectura Medallón.

Abstract

As part of the transition towards Industry 4.0, industrial assets and autonomous guided vehicles have evolved into systems capable of transmitting continuous streams of real-time telemetry, generating a vast amount of information. However, simply collecting this data does not, in itself, provide any operational value. Only through appropriate processing and analysis is it possible to transform this data into useful knowledge for strategic decision-making. This is where the idea for this project came from, developed in collaboration with EDAG, a leading organisation in the industrial sector.

Despite the progress made in predictive maintenance in recent years, there remains a significant gap between theoretical studies on anomaly detection and their effective application in production environments. This limitation is mainly due to the complexity of collecting high-quality data, the dynamic nature of industrial environments and the need to process information with low latency in order to take action before a failure occurs.

To address this issue, this project presents a comprehensive solution for real-time anomaly detection across a fleet of mobile robots. The system comprises a distributed infrastructure for continuous data ingestion and processing, a historical storage layer optimised for time series, and an analytics module based on machine learning for identifying anomalous behaviour. The system is rounded off by a visual monitoring and alerting tool for plant staff, developed using a clean, decoupled and scalable architecture.

Keywords: IIoT, Predictive Maintenance, Anomaly Detection, Machine Learning, AGV, AMR, Industrial Robots, Industry 4.0, Data Lakehouse, Medallion Architecture.

Agradecimientos

Resumen 1

Abstract 2

Agradecimientos 3

Nomenclature 10

1 Introduction 12

1.1 Motivation . . . 12

1.2 Objectives . . . 14

1.2.1 Industrial Context and Collaboration . . . 15

1.3 Scope and Limitations . . . 15

1.4 Methodology . . . 16

1.4.1 Domain Immersion and Preliminary Research . . . 16

1.4.2 Requirements Engineering and System Design . . . 16

1.4.3 Incremental Development and Validation . . . 17

1.5 Document structure . . . 18

2 Technologies 20

2.1 Git . . . 20

2.2 Bitbucket . . . 20

2.3 Jira . . . 20

2.4 Docker . . . 20

2.5 Visual Studio Code . . . 20

2.6 MQTT Broker: Mosquitto . . . 21

2.7 Diagrams: Draw.io . . . 21

2.8 Data Streaming Platform: Redpanda . . . 21

2.9 Telegraph . . . 21

2.10 Kafka Connect . . . 21

2.11 Apache Spark . . . 21

2.12 Apache Iceberg . . . 21

2.13 S3 Garage . . . 22

2.14 Time Series Database: QuestDB . . . 22

2.15 Grafana . . . 22

3 State of the Art 23

3.1 Taxonomy of Anomaly Detection Approaches for Industrial Telemetry . . . 23

3.1.1 Supervised, Unsupervised and Semi-supervised Paradigms . . . 23

3.1.2 Univariate versus Multivariate Modelling . . . 24
3.1.3 Static versus Temporal (Sequential) Models . . . 24
3.1.4 Methodological Selection for AGV/AMR Telemetry . . . 25
3.1.5 Architecture of the Gated Recurrent Unit Autoencoder . . . 26
3.1.6 Root-Cause Analysis and Explainability in Cyber-Physical Telemetry . . . 26
3.2 Scalable Real-Time Telemetry Pipelines and Data Lakehouse Architecture . . . 28
3.2.1 Physics-Informed Synthetic Telemetry and Digital Twin Emulation . . . 28
3.2.2 Domain Adaptation and Transfer Learning . . . 28
3.2.3 Real-Time Stream Ingestion and Decoupled Messaging Layer . . . 29
3.2.4 The Data Lakehouse Paradigm and Medallion Storage Model . . . 29
3.2.5 Transactional Table Management and Partitioning with Apache Iceberg . . . 29
3.2.6 High-Performance Time-Series Persistence and Operational Serving Layers . . . 30
4 Requirements Analysis and Specification 31
5 System Architecture Design 32
5.1 Introduction . . . 32
5.2 General Architectural Decisions . . . 33
5.2.1 Containerized Microservice Architecture . . . 33
5.2.2 Interface-Oriented Design . . . 34
5.2.3 Event-Driven Architecture . . . 34
5.2.4 Separation of Training and Inference Pipelines . . . 34
5.3 Source Systems Layer . . . 35
5.4 MQTT Broker Layer . . . 35
5.5 Event Streaming Layer . . . 35
5.5.1 Kafka provider selection . . . 36
5.5.2 Kafka Topic Design . . . 36
5.6 Streaming Integration and Inter-Service Connectivity . . . 38
5.6.1 MQTT to Event Streaming Ingestion Layer . . . 38
5.6.2 Streaming to Time-Series Persistence: Decoupled Kafka Connect for QuestDB . . . 39
5.6.3 Event Streaming to Analytical Processing Connectivity . . . 39
5.7 Stream & Processing Layer: Apache Spark . . . 39
5.7.1 Medallion Processing Pipeline and Job Topology . . . 40
5.8 Data Lakehouse Layer: Storage Architecture . . . 42
5.8.1 Object Storage Provider Selection: MinIO . . . 42
5.8.2 Table Format Selection: Apache Iceberg . . . 44
5.8.3 Automated Lakehouse Maintenance Architecture . . . 44
5.8.4 Bifurcated Storage Architecture: Operational vs. Analytical Persistence . . . 45

5.9 Machine Learning layer . . . 45
5.10 Observability Layer: Grafana Dashboards . . . 45
5.11 Security and Access Control Considerations . . . 45
6 System Implementation 46
6.1 Introduction . . . 46
6.2 Implementation Methodology . . . 46
6.2.1 Distributed Version Control and Commit Traceability . . . 46
6.2.2 Continuous Integration and Automated Verification . . . 47
6.3 Containerized Deployment . . . 47
6.3.1 Docker Compose Organization . . . 47
6.3.2 Network Segmentation . . . 48
6.3.3 Modular Service Configuration . . . 48
6.4 Source Systems: Vehicle Simulation and Physical Modeling . . . 49
6.4.1 Simulator Architecture and Containerized Deployment . . . 49
6.4.2 Motor and Drivetrain Dynamics . . . 50
6.4.3 Kinematics and Wheel Slip . . . 50
6.4.4 Thermal Dissipation Models . . . 51
6.4.5 Battery Equivalent Circuit Model (ECM) . . . 51
6.4.6 Physical Fault Injection and Transducer Emulation . . . 51
6.4.7 Synthesized Telemetry Channels and Payload Mapping . . . 52
6.4.8 AI-Assisted Implementation and Validation Workflow . . . 53
6.5 MQTT Broker Implementation and Integration . . . 53
6.5.1 Eclipse Mosquitto Deployment and Broker Configuration . . . 53
6.6 Event Stream Layer Implementation . . . 54
6.7 Streaming Integration and Inter-Service Connectivity . . . 56
6.7.1 Telemetry Bridging via Telegraf . . . 57
6.7.2 Kafka Connect Integration and Custom Single Message Transformations . . . 59
6.8 Stream Processing Layer: Apache Spark Implementation . . . 59
6.8.1 Docker Compose Service Definition . . . 59
6.8.2 Jobs . . . 63
6.8.3 Bronze Ingestion Job . . . 64
6.8.4 Silver Feature Engineering Job . . . 64
6.8.5 Gold Inference Job . . . 64
6.8.6 Lakehouse Maintenance Job . . . 64
6.9 Data Lakehouse Layer Implementation . . . 65
6.9.1 S3-Compatible Object Storage: MinIO . . . 65

6.9.2 Table Format Implementation: Apache Iceberg . . . 67
6.10 QuestDB Implementation and Integration . . . 67
6.10.1 Database Deployment and Schema Architecture . . . 67
6.10.2 Automated Lifecycle and Dependency Management . . . 68
6.11 Machine Learning Pipeline Implementation . . . 68
6.12 Grafana Implementation . . . 68
6.13 Security and Network Isolation Enforcement . . . 69
7 Testing and Evaluation 70
7.1 Introduction . . . 70
7.2 Testing: testing adjusted simulator . . . 72
7.2.1 Test Environment and Procedure . . . 72
7.2.2 Motor voltage and governing variables (Vm, Im, wm, Ra, La, Ke) . . . 72
7.2.3 Motor Torque and governing variables (τ_m, K_t, I_m) . . . 73
7.2.4 Mechanical torque balance and load dynamics (τ_m, τ_L, ω_m, B_m, J_m) . . . 74
7.2.5 Kinematic closure and wheel slip dynamics (v, ω_m, s, r_w, i_g) . . . 75
7.2.6 Motor thermal dynamics and power loss balance (T_m, T_{amb}, P_{joule}, P_{iron}, P_{cool}) . . . 76
7.2.7 Battery terminal voltage and internal-resistance drop (V_{bat}, V_{ocv}, R_{int}, I_{bat}) . . . 77
7.2.8 Interpretation and Limitations . . . 78
7.3 Evaluation: Data pipeline evaluation . . . 78
7.3.1 Evaluation Methodology and Workload Design . . . 79
7.3.2 Phase 1: Baseline Soak Test and Bottleneck Discovery (Day 1) . . . 79
7.3.3 Root-Cause Analysis and Lakehouse Pipeline Hardening . . . 81
7.3.4 Phase 2: Extended Multi-Day Validation Soak Test (Days 2–3) . . . 82
7.3.5 Consolidated Benchmark Summary and Conclusions . . . 85
8 Conclusions and Future Work 86
A Installation Manual 87
B User Manual 88
B.1 Application Service Ports and User Interfaces . . . 88
B.1.1 Internal Network Services . . . 88
Bibliography 90

1.1 Global AGV market forecast (2025–2034). Source: IMARC Group [17]. 13

3.1 Schematic architecture of the Gated Recurrent Unit Autoencoder (GRU-AE). A multivariate telemetry window is encoded by a GRU into a latent representation and subsequently reconstructed by a second GRU. The reconstruction residual is used for anomaly detection and channel-level attribution. 27

3.2 Figure X. Structure of a Gated Recurrent Unit (GRU). Source: fdeloche (2017), Wikimedia Commons, licensed under CC BY-SA 4.0. 27

5.1 High-level conceptual architecture of the end-to-end anomaly detection platform. 33

5.2 End-to-end integration topology connecting MQTT source systems, Redpanda streaming bus, QuestDB time-series storage via Kafka Connect, and Apache Spark analytics. 38

5.3 Data pipeline of the multi-stage Spark Structured Streaming architecture, showing dual-sink persistence to Redpanda topics and S3 Iceberg tables. 43

7.1 Motor voltage form, with all the terms 73

7.2 Motor torque form, with all the terms 74

7.3 Mechanical torque balance and load dynamics, with all the terms 75

7.4 Linear velocity form, with all the terms 76

7.5 Motor thermal dynamic and power loss balance, with all the terms 77

7.6 Battery tension form, with all terms 78

7.7 Evolution of memory footprint during the Phase 1 baseline soak test. While Spark, Redpanda, and QuestDB stabilized downward, MinIO exhibited runaway monotonic growth due to accumulating Iceberg metadata indices. 81

7.8 Evolution of memory footprint over the extended 38.9-hour multi-day soak test. MinIO, Spark, and QuestDB exhibited horizontal stabilization, while Redpanda reached its pre-allocated Seastar arena ceiling, proving zero unbounded resource growth. 84

3.1 Candidate models satisfying the unsupervised, multivariate and sequential requirements. 25

5.1 Kafka topic design, data flow, and partitioning key. 37

7.1 Experimental Host Hardware and Container Execution Environment 71
7.2 Container Resource Utilization and Memory Drift (Phase 1: Day 1 Baseline Soak Test) 80
7.3 Storage and Pipeline Performance Metrics Before vs. After Metadata Hardening 82
7.4 Container Resource Utilization and Memory Drift (Phase 2: Days 2–3 Extended 38.9-Hour Soak Test) 83
7.5 Consolidated Platform Metrics: Baseline Run vs. Hardened Multi-Day Soak Campaign 85

B.1 Application services, exposed ports, and user interface endpoints. 89

Nomenclature

AGV	Automated Guided Vehicle
AI	Artificial Intelligence
AMR	Autonomous Mobile Robot
CD	Continuous Deployment
CI	Continuous Integration
CM	Condition Monitoring
GC	Garbage Collector
GRU	Gated Recurrent Unit
GRU-AE	Gated Recurrent Unit Autoencoder
iForest	Isolation Forest
IIoT	Industrial Internet of Things
IoT	Internet of Things
JVM	Java Virtual Machine
LiDAR	Light Detection and Ranging
LOF	Local Outlier Factor
LSTM	Long Short-Term Memory
LSTM-AE	Long Short-Term Memory Autoencoder
ML	Machine Learning
MQTT	Message Queuing Telemetry Transport
PdM	Predictive Maintenance
PoC	Proof of Concept
RCA	Root Cause Analysis

SIMD Single Instruction, Multiple Data

SLAM Simultaneous Localization and Mapping

SOC State of Charge (battery)

SQL Structured Query Language

SSM State-Space Model

TSDB Time Series Database

VDA 5050 Verband der Automobilindustrie 5050

XAI Explainable Artificial Intelligence

1 Introduction

1.1 Motivation

Industry 4.0 is no longer just a concept, but a reality transforming factories worldwide. The arrival of these emerging technologies opens up vast opportunities to optimize production processes, shifting the focus from mere automation toward creating truly intelligent factories. A key enabler of Industry 4.0 is the deployment of Automated Guided Vehicles (AGVs) and Autonomous Mobile Robots (AMRs).

AGVs are driverless vehicles used to automate material transportation in warehouses and manufacturing facilities. They follow predefined routes using physical or digital guidance systems such as magnetic tapes, embedded wires, floor markings, QR codes, or laser reflectors. Common industrial tasks performed by AGVs include:

- Supplying raw materials to production lines.
- Transporting semi-finished products between workstations.
- Delivering finished goods to warehouse storage areas.
- Supporting pallet handling and warehouse logistics.

AMRs represent a more advanced class of driverless vehicles. They navigate dynamically using onboard sensors and technologies such as cameras, LiDAR, and SLAM (Simultaneous Localization and Mapping), allowing them to adapt their routes according to changing environmental conditions without requiring physical floor infrastructure. Common production applications include:

- Autonomous material delivery between production cells.
- Just-in-time supply of components to assembly stations.
- Flexible transport of tools, fixtures, and kitting carts.
- Internal logistics in facilities with frequently changing layouts.

- Collaborative transportation tasks alongside human operators.

As seen in Figure 1.1, their adoption across industries continues to surge, with market research pointing to sustained, strong growth in the coming years [17].

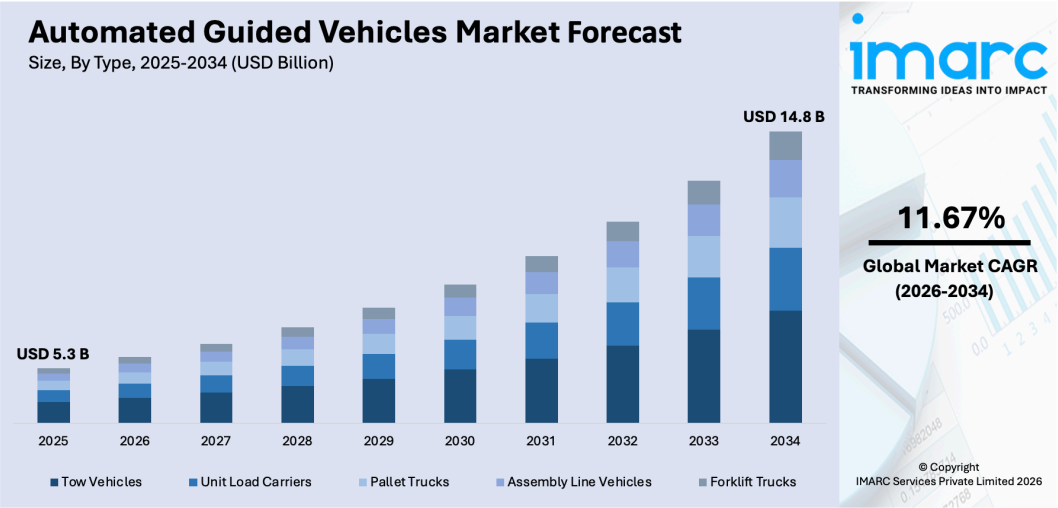


Figure 1.1. *Global AGV market forecast (2025–2034). Source: IMARC Group [17].*

Because these vehicles play a vital role in production, any failure can paralyze the assembly line, resulting in substantial financial losses. Studies reveal that unexpected downtime can range from $36,000 per hour in fast-moving consumer goods to upwards of $2.3 million per hour in the automotive sector [35]. Consequently, preventing such disruptions is not merely an operational challenge, but a critical financial priority for modern manufacturers.

A promising solution to this challenge is predictive maintenance (PdM) using machine learning. Unlike reactive maintenance (repairing equipment post-breakdown) or scheduled preventive maintenance (servicing assets at arbitrary calendar intervals), predictive maintenance leverages continuous condition monitoring (CM) and operational sensor telemetry to forecast impending asset failures. By anticipating faults before they manifest, this approach can significantly reduce vehicle failures and the associated costs of unplanned downtime.

The literature on machine learning for anomaly detection is extensive, however, a significant gap remains between theoretical studies and real-world implementation, as researchers encounter major hurdles when scaling proposed solutions to production environments [24].

Through a review of this literature, four key limitations emerge as critical factors when developing a real-time anomaly detection system for AGVs and AMRs [4]:

- **High-volume data management and processing:** Managing and automatically processing vast amounts of data, ensuring not only scale handling but also the quality and relevance of the ingested information.
- **Dynamism of industrial environments:** The inherently dynamic nature of manufacturing settings requires continuous adaptation to shifting conditions and novel anomaly types.

- **Ambiguity of anomalies:** The challenge of accurately classifying anomalies to establish a robust mapping between newly detected anomalies and known failure modes.
- **Real-time integration of machine learning models:** Deploying machine learning inference within real-time environments to enable immediate anomaly detection and automated response.

To complement the core challenges identified in the literature, an additional constraint must be considered in this work: the total lack of labeled historical error data, which precludes the use of supervised learning approaches.

1.2 Objectives

As demonstrated in the previous section, AGVs and AMRs are fully integrated into Industry 4.0 processes. While they drive automation across the production line, their critical role implies that any asset failure can lead to a complete halt in production. However, while numerous studies validate the effectiveness of machine learning for anomaly detection, translating these approaches into real-world production environments presents significant technical limitations.

To address these challenges, this project aims to design and implement a fully decoupled, end-to-end architecture for real-time anomaly detection. Broadly speaking, the proposed pipeline encompasses the entire data lifecycle: from simulated AGV telemetry generation, real-time data ingestion, processing, and persistence, to unsupervised machine learning anomaly detection and interactive visualization of diagnostic metrics.

To achieve this overarching goal, the project is structured into five specific technical objectives:

1. **Adaptation of the VDA 5050[37] Simulator[5]:** Due to hardware constraints and limited access to physical AGVs, utilizing real industrial data is often unfeasible. Therefore, a Rust-based VDA 5050 simulator is to be adapted to generate realistic telemetry streams. The simulation is extended beyond pure protocol compliance to model real-world robot physics and key health metrics, including motor current, motor voltage, motor temperature, linear velocity, and battery state. Generative AI tool(Claude Sonnet 5.0) is to be leveraged to assist in extending the Rust codebase.
2. **Real-Time Data Pipeline Implementation:** To process raw telemetry streams before they reach downstream machine learning components, Apache Spark[1] is used to ingest raw streams, cleanse null fields, append ingestion timestamps, and standardize data structures. Cleaned events are to be published to a Redpanda[30] event broker. This Kafka-compatible streaming layer decouples system modules, allowing independent producers and consumers to interact seamlessly.
3. **Data Lakehouse and Medallion Architecture[8]:** To ensure scalable, long-term persistence, a Data Lakehouse paradigm is adopted. It combines the cost-effective storage of object repositories (S3-compatible Garage bucket[7]) with the structured, ACID-compliant query capabilities of Apache Iceberg[15]. Data is to be organized following a Medallion Architecture:
    - **Bronze Layer:** To store raw, unprocessed telemetry streams exactly as received.
    - **Silver Layer:** To contain cleaned, validated, and enriched data with added ingestion metadata.
    - **Gold Layer:** To hold business-level aggregations, detected anomalies, and analytical outputs.

4. **Deep Learning for Anomaly Detection:** Serving as the core analytical engine, this component focuses on unsupervised anomaly detection using deep learning frameworks (PyTorch). Following a review of current methodologies, a deep learning model (e.g., Autoencoder) is to be deployed to detect subtle telemetry deviations. Furthermore, the objective extends beyond raw mathematical score calculation to map detected anomalies to clear, non-ambiguous diagnostic labels for human interpretability.

5. **Visualization and Monitoring Layer:** To present actionable insights to operational staff, telemetry and anomaly scores are to be persisted in QuestDB[27], a high-performance time-series database. Grafana[16] dashboards then query QuestDB to display real-time metrics, system health status, and historical anomaly alerts.

This multi-layered approach not only supports live monitoring, but through the Medallion Architecture and Iceberg time-travel capabilities, it allows historical raw data (Bronze layer) to be reprocessed whenever model architectures or machine learning requirements evolve.

1.2.1 Industrial Context and Collaboration

This project is carried out in collaboration with EDAG Engineering Group. As an international engineering service provider for the automotive and manufacturing industries, EDAG actively develops cutting-edge Industry 4.0 solutions. Developing this project during an internship at EDAG provides direct insight into real-world industrial environments, system requirements, and deployment constraints. Specifically, this work contributes to the **Metys** project [12], an initiative aimed at optimizing software development and smart production workflows.

1.3 Scope and Limitations

The scope of this project encompasses the end-to-end design and implementation of a real-time anomaly detection system for Autonomous Guided Vehicles (AGVs) and Autonomous Mobile Robots (AMRs).

It is important to emphasize that this work does not focus on prescribing automated corrective actions or actively preventing anomalies; rather, it is restricted to their detection and operational visualization. Consequently, determining the appropriate mitigation strategies remains at the discretion of the end user or plant operator.

Furthermore, the operational scope of this project is subject to several specific constraints and limitations:

- **Industrial Environment Constraints:** The development process was conducted in close collaboration with an industrial partner. While this provided access to real-world operational contexts and expert technical support, it also imposed fixed technical requirements. Specifically, non-functional constraints—such as adhering to the VDA 5050 protocol standard [37]—were mandatory to ensure seamless integration into the target industrial platform [12].

- **Dependency on Simulated Telemetry and Anomaly Data:** Due to the unavailability of physical robot fleets for continuous data acquisition, the Machine Learning models are trained exclusively on telemetry generated by simulated AGV/AMR fleets. Consequently, the model may struggle to fully generalize to the complex stochastic behavior inherent to physical production environments. Furthermore, because real failure logs were unavailable, performance evaluation is conducted using synthetically injected anomalies. While testing on a physical AMR platform remains a prospective goal,

replicating complex mechanical and electrical anomalies on actual hardware carries significant economic and operational costs.

- **Context Sensitivity and False Positives:** Unsupervised Machine Learning models rely on establishing a nominal baseline from the assets' operational telemetry. Consequently, any valid operational state or contextual change omitted from the training dataset—such as an unseen payload weight being transported or an unfamiliar speed setting—will inevitably be misclassified as an anomaly during inference, resulting in false positives.
- **Computational Constraints:** System throughput and real-time processing capabilities are bounded by the hardware resources (CPU, RAM, and storage) of the local host machine used during development. As a result, this prototype does not fully capture the performance dynamics or dynamic scalability of a cloud cluster deployment.
- **Resource and Time Constraints:** Due to the fixed timeframe and individual nature of an academic thesis, the project scope was intentionally bounded to deliver a fully functional Proof of Concept (PoC). Consequently, exhaustive experimental exploration of complex MLOps pipelines and alternative model architectures was beyond the achievable scope.

1.4 Methodology

The methodological approach adopted in this project combines a traditional plan-driven software engineering process for requirements definition and system design with an agile development framework based on Scrum[34] for implementation, integration, and validation activities. This hybrid approach was selected due to the collaborative nature of the project with EDAG and the need to balance clearly defined industrial requirements with the flexibility required during the development of Machine Learning and Industrial Internet of Things (IIoT) solutions.

The methodology is structured into three main phases: (i) domain immersion and preliminary research, (ii) requirements engineering and system design, and (iii) incremental development and validation.

1.4.1 Domain Immersion and Preliminary Research

Before the technical development phase, the student participated in activities related to an industrial project within EDAG involving Automated Guided Vehicles (AGVs) and Autonomous Mobile Robots (AMRs). This initial period was essential to understand the operational context, industrial processes, and technological challenges associated with smart manufacturing environments. In parallel, an extensive literature review was conducted to establish the scientific and technical foundations of the project. Research papers, industrial reports, and technical documentation were analyzed to identify state-of-the-art solutions in Industrial IoT architectures, streaming data platforms, anomaly detection techniques, and data engineering infrastructures for real-time analytics. This phase enabled the acquisition of domain-specific knowledge, facilitated technology selection, and provided a deeper understanding of the practical requirements and constraints that would later influence architectural and design decisions.

1.4.2 Requirements Engineering and System Design

The requirements engineering phase followed a plan-driven approach. Unlike conventional software projects in which requirements emerge progressively from a product backlog, the initial specifications of this work were provided directly by EDAG

within the framework of the ILTIS project. These specifications included business objectives, technical requirements, industrial constraints, and interoperability standards, particularly the VDA 5050 protocol used for communication with AGVs and AMRs. The availability of these requirements from the beginning allowed the project scope to be clearly defined and the architectural foundations of the data pipeline to be established before implementation activities started. In addition, the information gathered during the literature review was used to evaluate existing solutions, identify technological limitations, and justify he choice of specific frameworks and echnologies adopted throughout the project. The outcome of this phase included:

- Definition of functional and non-functional requirements.
- Identification of industrial constraints and communication standards.
- Design of the data architecture and processing pipeline.
- Establishment of technical acceptance criteria.
- Selection of technologies and tools.

1.4.3 Incremental Development and Validation

The implementation phase was conducted using Scrum[34], an agile framework widely adopted in both software engineering and data engineering projects and actively used within EDAG. This methodological alignment facilitated integration into the company's working practices while supporting efficient knowledge transfer between academic and industrial stakeholders. The adoption of Scrum was particularly suitable due to the exploratory nature of several project components. Integrating real-time IIoT infrastructures with Machine Learning models introduces uncertainty regarding telemetry formats, data quality, feature extraction strategies, and model performance. Scrum addresses these challenges through short development iterations that support continuous feedback, progressive refinement, and incremental delivery of functionalities.

1.4.3.1 Role Adaptation

Since the project was carried out by a single student, Scrum roles were adapted accordingly:

- **Product Owner**: Assumed jointly by the academic supervisors from the University of Málaga and the designated supervisor at EDAG. Their responsibilities included defining objectives, prioritizing functionalities, validating deliverables, and ensuring alignment with business requirements.
- **Scrum Master and Development Team**: Assumed by the student, responsible for sprint planning, software development, infrastructure deployment, Machine Learning experimentation, integration activities, testing, and documentation.

1.4.3.2 Sprint Planning

The development process was organized into two-week sprints. The initial roadmap consisted of the following iterations:

1. IIoT Simulation Environment Setup.
2. Streaming Pipeline and Data Ingestion.

3. Data Persistence and Lakehouse Architecture.
4. Data Infrastructure Validation and Performance Testing.
5. Artificial Intelligence and Anomaly Detection Module I.
6. Artificial Intelligence and Anomaly Detection Module II.
7. Observability, Alerting, and Final Validation.

At the end of each sprint, results were reviewed and evaluated against the predefined acceptance criteria. Feedback obtained during these reviews was incorporated into subsequent iterations, enabling continuous improvement of both the infrastructure and the Machine Learning components throughout the project lifecycle.

1.5 Document structure

This document is organized into eight main chapters, including this introductory chapter, followed by two supplementary appendices that provide practical guides for system deployment and operation:

1. **Introduction (Chapter 1):** Provides an overview of the industrial motivation within Industry 4.0 and autonomous mobile robotics, formalizes the overarching and specific technical objectives of the project, establishes the industrial collaboration with EDAG and the Metys project, delimits the operational scope and constraints, details the hybrid software engineering methodology (combining plan-driven design with agile Scrum sprints), and outlines the structural organization of this work.
2. **Technologies (Chapter 2):** Offers a comprehensive summary of the technological stack, libraries, and frameworks employed across the development lifecycle, spanning container orchestration, edge IoT messaging protocols (MQTT and Mosquitto), distributed event streaming, Data Lakehouse technologies, and deep learning platforms.
3. **State of the Art (Chapter 3):** Establishes the theoretical foundation and literature review supporting this research, covering deep anomaly detection paradigms, recurrent autoencoder architectures (GRU-AE), explainable root-cause analysis via residual decomposition, physics-informed synthetic telemetry generation, and scalable real-time Data Lakehouse paradigms structured around the Medallion architecture and Apache Iceberg.
4. **Requirements Analysis and Specification (Chapter 4):** Need to be done.
5. **System Architecture Design (Chapter 5):** Need to be done.
6. **System Implementation (Chapter 6):** Need to be done.
7. **Testing and Evaluation (Chapter 7):** Need to be done.
8. **Conclusions and Future Work (Chapter 8):** Synthesizes the achieved results and key findings relative to the initial technical objectives, reviews lessons learned from the industrial collaboration, and outlines prospective lines of future research and technical enhancements.

9. **Appendices (Appendices A and B):** Contain supporting operational documentation, comprising Appendix A (Installation Manual), which details environment prerequisites, container orchestration, and deployment steps, and Appendix B (User Manual), which provides instructions for sending orders and instant actions, simulating anomalies, monitoring Grafana dashboards, and inspecting real-time telemetry.

2 Technologies

2.1 Git

Git is a free, open source distributed version control system. It prioritises speed and efficiency, handling projects of any size while keeping the complete history of every change made to the code base [14].

Due to the continuous deployment nature of the project, this was the most suitable version control system.

2.2 Bitbucket

Bitbucket is a Git-based code hosting and collaboration platform built for development teams. It brings together code review, automated testing and confident deployment in a single place, and it integrates natively with Jira and Trello so that the whole software team can coordinate around a shared project [2].

The industrial partner uses Bitbucket internally, so its adoption was treated as a non-functional requirement of the project.

2.3 Jira

Jira is a project management tool that helps teams of any size plan, track and deliver work. It allows breaking large goals into manageable tasks, clarifying who is doing what and by when, and keeping everyone aligned around a single source of truth [3].

This tool is useful for the student to keep a clear map of the whole development and writing process of the system.

2.4 Docker

Docker is a suite of products that leverages operating-system-level virtualization to deliver software in lightweight packages called containers. It automates application deployment so that workloads run consistently across different computing environments [39].

This allowed the system to be split into well differentiated microservices, each packaged and deployed independently.

2.5 Visual Studio Code

Visual Studio Code is an editor developed by Microsoft, available on Windows, Linux, macOS and web browsers. It provides debugging support, syntax highlighting, intelligent code completion, snippets, code refactoring and built-in version control through

Git. Its look and behaviour can be tailored through themes, keyboard shortcuts and preferences, and its functionality can be extended with plugins, including turning it into a full IDE for other programming languages [40].

2.6 MQTT Broker: Mosquitto

An MQTT message broker is the central component that enables communication between clients and IoT devices over the MQTT protocol. It works on a publish-subscribe (pub/sub) messaging model: the broker receives messages from publishers, validates their publishing rights, queues them according to their Quality of Service (QoS) level and routes them to the authorized subscribers.

Eclipse Mosquitto is an open source (EPL/EDL licensed) message broker implementing MQTT protocol versions 5.0, 3.1.1 and 3.1. Its lightweight footprint makes it suitable for any device, from low power single board computers to full servers [11].

2.7 Diagrams: Draw.io

Draw.io is an open source graph drawing application. It is used to translate the ideas and the architecture of the system into diagrams that can be documented on paper [10].

2.8 Data Streaming Platform: Redpanda

Redpanda is a data streaming platform compatible with the Apache Kafka API[29] that delivers fault-tolerant message streaming with a simpler architecture and lower latency. In this project it acts as the central event bus that decouples the producers of telemetry data from the downstream consumers [30].

2.9 Telegraph

To do

2.10 Kafka Connect

Kafka Connect is a framework and toolset for building and running data pipelines between Apache Kafka® and other data systems. It provides a scalable and reliable way to move data in and out of Kafka. It has reusable plugins that you can use to stream data between various external Kafka systems[31].

2.11 Apache Spark

Apache Spark is a unified analytics engine for large-scale data processing. It offers high-level APIs in Java, Scala, Python and R on top of an optimized engine capable of executing general execution graphs. Its ecosystem includes Spark SQL for SQL and structured data processing, the pandas API on Spark for pandas workloads, MLlib for machine learning, GraphX for graph processing, and Structured Streaming for incremental computation and stream processing [1].

2.12 Apache Iceberg

Apache Iceberg is an open table format built for large analytical datasets stored in data lakes. Instead of treating the lake as a plain collection of files, Iceberg manages them as real tables, adding database-like capabilities such as schema evolution, time travel and more efficient query planning on top of object storage. This improves the reliability, performance and flexibility of modern data architectures, letting analysts, data scientists and engineers work with data lake data more easily [15].

2.13 S3 Garage

Garage is a lightweight, geo-distributed data store that implements the Amazon S3 object storage protocol. It lets applications store large blobs such as pictures, videos or documents in a redundant multi-node setup, and the S3 API is versatile enough to also serve static websites [7].

This technology provides the storage layer where the Iceberg tables will be persisted, enabling the construction of the Data Lakehouse.

2.14 Time Series Database: QuestDB

QuestDB is an open source time-series database engineered for low latency. Its column-oriented, time-partitioned storage engine, together with memory-mapped files and vectorized (SIMD) execution, supports high-throughput ingestion and millisecond-level analytical queries. SQL is extended with time-series operators such as SAMPLE BY, LATEST ON and ASOF JOIN [27].

Due to the nature of the telemetry data, this database is optimal for the purpose of the application and for connecting with the observation layer, Grafana.

2.15 Grafana

Grafana open source software allows querying, visualizing, alerting on and exploring metrics, logs and traces wherever they are stored. It provides the tools to turn time-series database (TSDB) data into insightful graphs and visualizations, and its plugin framework can also connect other data sources such as NoSQL/SQL databases, ticketing tools like Jira or ServiceNow, and CI/CD tooling like GitLab [16].

This enables the observation part of the application, acting as the end application that the worker will see.

3

State of the Art

Developing an end-to-end predictive maintenance system for fleets of Autonomous Guided Vehicles (AGVs) and Autonomous Mobile Robots (AMRs) requires synthesizing research from two core domains: deep learning-based anomaly detection and scalable real-time data engineering. While machine learning algorithms provide the analytical capacity to identify subtle operational degradation in multivariate sensor streams, their practical viability in industrial cyber-physical systems relies entirely on a robust streaming infrastructure capable of ingesting, transforming, and persisting high-frequency telemetry with low latency and transactional reliability [21, 22]. This chapter reviews the theoretical foundations and current literature supporting both pillars. Section 3.1 examines the taxonomy of deep anomaly detection methodologies, motivating the selection of Gated Recurrent Unit Autoencoders (GRU-AE). Section 3.2 explores physics-informed synthetic data generation, scalable stream ingestion architectures, event-driven decoupling, the Data Lakehouse paradigm structured via the Medallion architecture, and high-performance time-series persistence for real-time operational monitoring [21, 22].

3.1 Taxonomy of Anomaly Detection Approaches for Industrial Telemetry

Developing a practical anomaly detection engine for AGV and AMR fleets requires a clear understanding of the main design axes that govern algorithmic choice. Rather than adopting a purely representation-learning taxonomy [25], this section organises the literature along three orthogonal dimensions that directly map onto the characteristics of industrial telemetry: (i) the availability of labelled data (supervised, unsupervised and semi-supervised paradigms), (ii) the treatment of sensor variables (univariate versus multivariate modelling), and (iii) the handling of temporal structure (static versus sequential models). This problem-oriented classification provides a more transparent rationale for the methodological decisions taken in the present work.

3.1.1 Supervised, Unsupervised and Semi-supervised Paradigms

Anomaly detection methods can first be distinguished by the degree of supervision they require.

- **Supervised approaches** assume the availability of labelled normal and anomalous samples. Classical classifiers (Random Forests, Gradient Boosting, fully-connected or convolutional networks) can then be trained to discriminate the two classes. While highly effective when sufficient labelled faults exist, this setting is rarely realistic in industrial predictive maintenance: true failures are scarce, expensive to annotate, and their statistical distribution drifts over time as equipment ages or operating conditions change.

- **Unsupervised approaches** learn solely from nominal data and flag observations that deviate from the learned normal pattern. Reconstruction-based models (autoencoders), one-class classification methods (Deep SVDD [33]), and density-estimation techniques belong to this family. Because labelled anomalies are unnecessary, unsupervised methods dominate industrial deployments.

- **Semi-supervised approaches** occupy an intermediate position: they are trained predominantly on normal data, optionally incorporating a small number of known anomalies when available. Many practical systems fall into this category, as a limited set of historically confirmed faults can be used to refine decision thresholds or to regularise the model.

In the context of AGV/AMR fleets, the virtual absence of comprehensive labelled failure data makes unsupervised and semi-supervised formulations the only viable options.

3.1.2 Univariate versus Multivariate Modelling

A second fundamental distinction concerns the way sensor channels are treated.

- **Univariate methods** analyse each telemetry variable independently. Statistical control charts, seasonal-trend decomposition, or per-channel autoencoders are typical examples. These approaches are simple to implement and easy to interpret, yet they ignore cross-variable correlations that often carry the most discriminative information about mechanical or electrical degradation.

- **Multivariate methods** explicitly model the joint behaviour of multiple sensors (motor current, voltage, temperature, linear velocity, battery state-of-charge, etc.). By capturing inter-variable dependencies they can detect anomalies that remain invisible when channels are examined in isolation—for instance, an abnormal rise in temperature accompanied by an unexpected drop in current. Representative techniques include multivariate autoencoders, graph neural networks that treat sensors as nodes, and models that learn a joint latent distribution over the whole telemetry vector.

Given that AGV and AMR platforms generate tightly coupled multi-sensor streams, multivariate modelling is essential for high detection sensitivity and for meaningful root-cause attribution.

3.1.3 Static versus Temporal (Sequential) Models

The third axis addresses the presence or absence of explicit temporal modelling.

- **Static (snapshot) models** operate on individual observations or on fixed-size windows treated as independent samples. Classical one-class SVM, Isolation Forest, or feed-forward autoencoders fall into this group. They are computationally light but cannot exploit the sequential dynamics that characterise electromechanical systems.

- **Temporal (sequential) models** maintain an internal state that evolves over time, thereby capturing short- and medium-term dependencies. Recurrent architectures (LSTM, GRU), temporal convolutional networks, Transformers, and modern state-space models (including Mamba) belong to this family. Because the physical behaviour of motors, batteries and drivetrains is inherently dynamic, temporal models generally outperform static counterparts on industrial telemetry.

3.1.4 Methodological Selection for AGV/AMR Telemetry

Evaluating the three dimensions discussed above against the operational constraints of real-time fleet monitoring yields a clear set of requirements:

- **Supervision**: the scarcity of labelled faults dictates an unsupervised (or at most semi-supervised) formulation.
- **Variable interaction**: the strong physical coupling among sensors requires a multivariate model.
- **Temporal structure**: the continuous evolution of electromechanical states necessitates a sequential architecture with memory.

These three constraints significantly narrow the space of candidate algorithms. Table 3.1 summarises representative models that satisfy all three requirements and have been applied to industrial or multivariate time-series anomaly detection.

Table 3.1. *Candidate models satisfying the unsupervised, multivariate and sequential requirements.*

Model family	Representative works	Main characteristics
LSTM / GRU Autoencoders	OmniAnomaly [36], InterFusion [19]	Strong temporal modelling, channel-wise residual interpretability, moderate computational cost
Variational Recurrent models	OmniAnomaly, InterFusion	Probabilistic reconstruction, good uncertainty handling
Temporal Convolutional Networks (TCN)	MAD-TCN [6]	Parallelisable, long receptive fields, less natural residual attribution
Transformer-based	Anomaly Transformer [41]	Excellent long-range dependencies, higher computational cost and latency
State-Space Models (SSMs)	MambaAD [26]	Linear-time sequence modelling, strong long-range capacity, emerging in industrial applications

Among these candidates, the **Gated Recurrent Unit Autoencoder (GRU-AE)** was selected as the primary detection engine for the following reasons:

- **Accuracy–latency trade-off**: GRU networks retain most of the temporal modelling capacity of LSTMs while using fewer parameters and gates, resulting in faster training and lower inference latency [9]. This is critical for continuous execution inside a streaming Silver-layer pipeline.
- **Native channel-level explainability**: the reconstruction residual can be decomposed per sensor with negligible extra cost, providing immediate and intuitive root-cause indications for maintenance personnel.

- **Proven industrial performance**: models such as OmniAnomaly and InterFusion have demonstrated strong results on real multivariate telemetry while supporting variable-level interpretation.
- **Implementation and deployment simplicity**: mature libraries, straightforward quantisation possibilities, and easier integration into existing stream-processing frameworks compared with more recent architectures (Transformers or SSMs).

While the GRU-AE constitutes the core operational detector, the framework also contemplates the integration of modern State-Space Models (in particular Mamba-style architectures) as a complementary benchmark. These models offer linear-time complexity with respect to sequence length and improved long-range dependency modelling, which may prove advantageous for longer prediction horizons or more complex degradation patterns. Their evaluation alongside the GRU-AE allows a controlled comparison between a mature, production-ready baseline and a state-of-the-art sequential alternative.

3.1.5 Architecture of the Gated Recurrent Unit Autoencoder

The Gated Recurrent Unit Autoencoder (GRU-AE) combines two complementary ideas: the reconstruction principle of autoencoders and the temporal modelling capacity of recurrent neural networks. This subsection provides a concise explanation of its internal structure.

An autoencoder is a neural network trained to reconstruct its own input. It consists of two parts:

- an **encoder** that compresses the input into a lower-dimensional latent representation, and
- a **decoder** that attempts to recover the original input from that latent representation.

When the model is trained exclusively on nominal data, it learns to reconstruct normal patterns with low error. Observations that deviate from these patterns produce high reconstruction error and can therefore be flagged as anomalous.

Because industrial telemetry is sequential, the encoder and decoder are implemented with Gated Recurrent Units (GRUs) rather than feed-forward layers. A GRU maintains an internal hidden state that evolves over time, allowing the network to capture short- and medium-term temporal dependencies.

Figure 3.1 illustrates the overall architecture and Figure 3.2 the unit GRU structure. A multivariate telemetry window

$$\mathbf{X}_t = \left(\mathbf{x}_{t-T+1}, \ldots, \mathbf{x}_t\right)^\top \in \mathbb{R}^{T \times M}$$

is fed to the encoder GRU, which processes the sequence step by step and produces a latent representation. The decoder GRU then reconstructs the window, yielding $\hat{\mathbf{X}}_t$. The difference between the original and reconstructed signals forms the basis for anomaly scoring (detailed in the next subsection).

This design offers a favourable balance between temporal modelling power, computational efficiency and interpretability, making it well suited for real-time deployment inside a streaming data pipeline.

3.1.6 Root-Cause Analysis and Explainability in Cyber-Physical Telemetry

While deep autoencoding architectures effectively identify temporal deviations in high-dimensional telemetry, a scalar anomaly score alone offers limited operational value to maintenance technicians [36, 19]. In industrial predictive maintenance, detection must be accompanied by Root-Cause Analysis (RCA) mechanisms that help isolate the most affected subsystems.

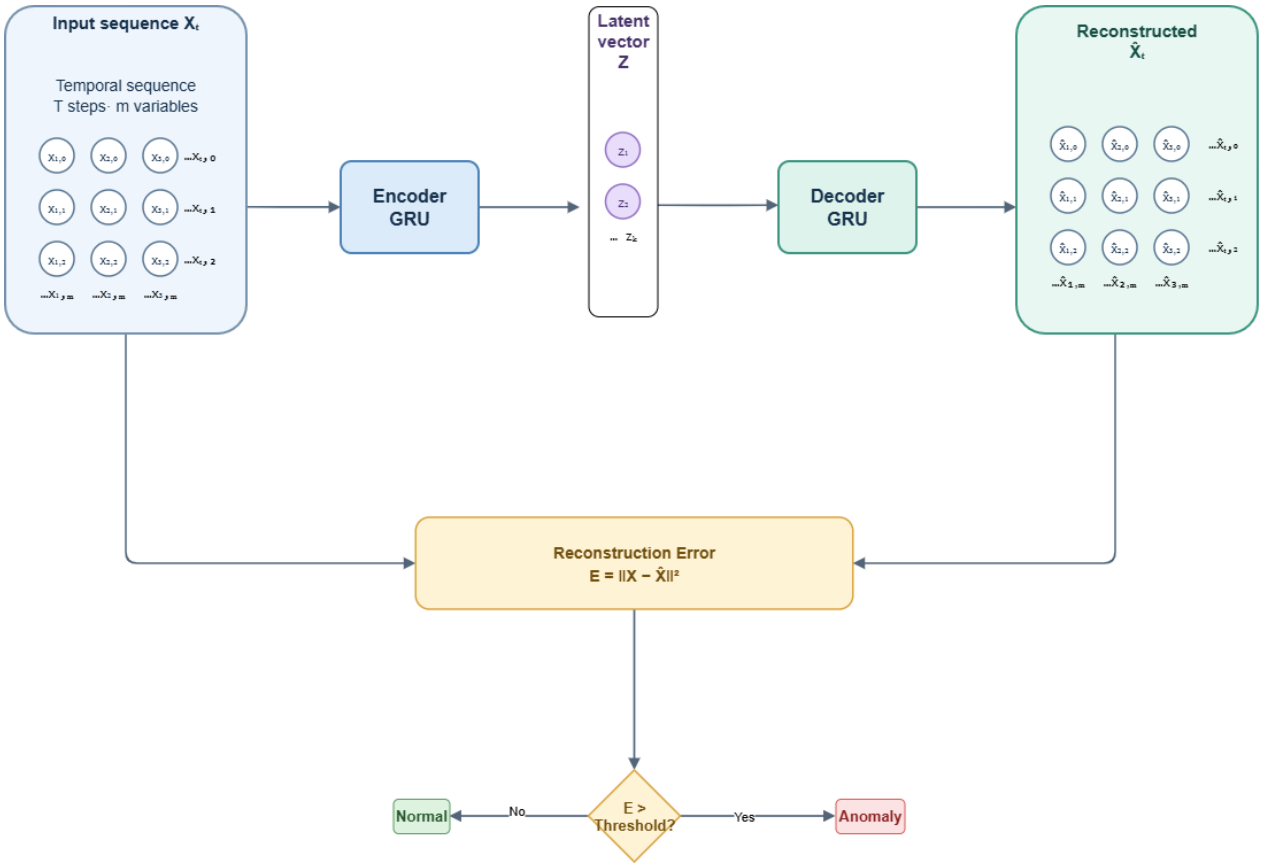


Figure 3.1. *Schematic architecture of the Gated Recurrent Unit Autoencoder (GRU-AE). A multivariate telemetry window is encoded by a GRU into a latent representation and subsequently reconstructed by a second GRU. The reconstruction residual is used for anomaly detection and channel-level attribution.*

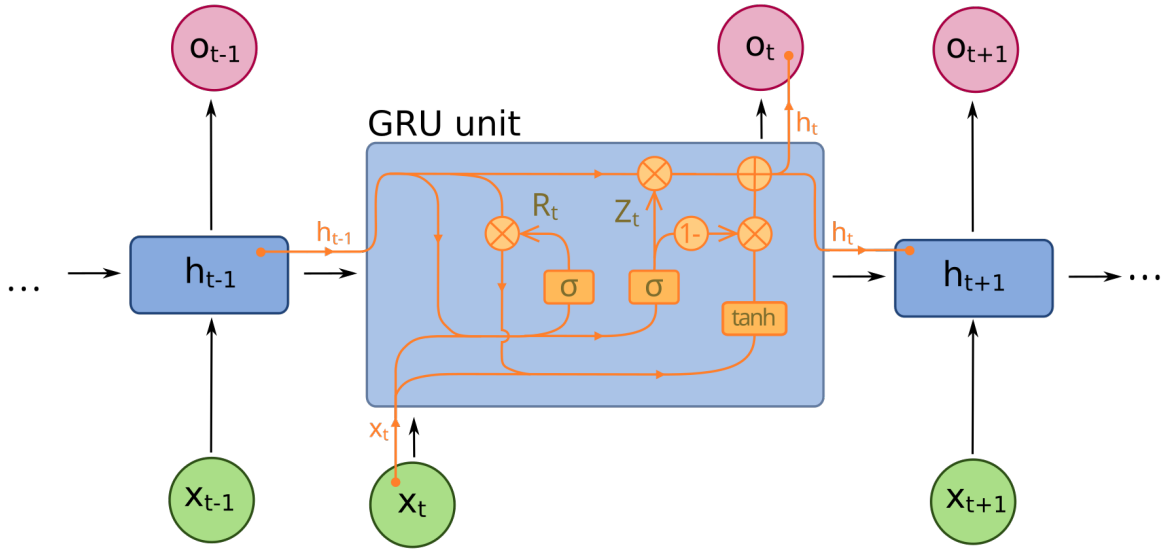


Figure 3.2. *Figure X. Structure of a Gated Recurrent Unit (GRU). Source: fdeloche (2017), Wikimedia Commons, licensed under CC BY-SA 4.0.*

Reconstruction-based models address this need through channel-level residual decomposition. Let $\mathbf{x}_t = (x_{t,1}, \ldots, x_{t,M})^\top \in \mathbb{R}^M$ denote the telemetry vector at time t and $\hat{\mathbf{x}}_t$ its reconstruction. The per-channel reconstruction error is defined as

$$e_{t,m} = \big(x_{t,m} - \hat{x}_{t,m}\big)^2, \qquad m = 1, \ldots, M, \tag{3.1}$$

and the total residual decomposes exactly as

$$\|\mathbf{x}_t - \hat{\mathbf{x}}_t\|_2^2 = \sum_{m=1}^{M} e_{t,m}. \tag{3.2}$$

An observation is declared anomalous when the total residual exceeds a threshold τ calibrated on nominal data:

$$a_t = \mathbb{I}\Big[\sum_{m=1}^{M} e_{t,m} > \tau\Big]. \tag{3.3}$$

By ranking the individual contributions $e_{t,m}$, the system provides immediate variable-level attribution. For example, simultaneous elevations in motor temperature and current residuals typically indicate electromechanical friction or overload, whereas an isolated deviation in battery state-of-charge points toward cell degradation. Mapping these residual signatures to domain-specific rules converts the continuous model outputs into actionable diagnostic labels.

It should be noted that residual decomposition yields a form of local attribution rather than a full causal analysis. In complex multi-fault scenarios, complementary techniques may be required; nevertheless, channel-wise residuals remain one of the most practical and computationally efficient explainability mechanisms available for real-time industrial telemetry.

3.2 Scalable Real-Time Telemetry Pipelines and Data Lakehouse Architecture

The implementation of modern data architectures in industrial mobile robotics requires an end-to-end approach spanning from high-speed telemetry generation and ingestion to low-cost analytical storage and real-time operational exposure. This section examines the state of the art in scalable data pipeline design, evaluating the transition from physics-informed digital twin data generation to structured storage via the Data Lakehouse paradigm and high-availability time-series databases.

3.2.1 Physics-Informed Synthetic Telemetry and Digital Twin Emulation

The limited availability of real-world operational data and failure records is a major challenge when developing machine learning pipelines for industrial mobile robotics. Physics-informed Digital Twins and synthetic telemetry generators can help address this limitation by providing representative data for training and evaluating anomaly detection models [20, 32].

At the communication layer, the open-source Rust-based `vda5050_vehicle_simulator` [5] provides a means of emulating VDA 5050 communication traffic. Such infrastructure can facilitate reproducible testing of telemetry ingestion pipelines and anomaly detection models under different operational scenarios and simulated fault conditions.

3.2.2 Domain Adaptation and Transfer Learning

Despite the benefits of synthetic telemetry, models trained exclusively on simulated data may experience performance degradation when deployed on physical robots due to the *sim-to-real domain gap*, which arises from differences in sensor noise, vehicle dynamics, and operating conditions. Combining synthetic and real-world data can help improve model performance under certain conditions [18].

Several techniques could mitigate this gap, including fine-tuning a pretrained GRU autoencoder on real-world telemetry, recalibrating reconstruction error thresholds, and applying domain adaptation methods such as Domain-Adversarial Neural Networks (DANN) [38]. These approaches could help adapt learned temporal representations to real industrial environments while reducing the amount of labeled real-world data required.

Due to time and computational hardware limitations, these techniques are not implemented or experimentally evaluated in this project. Instead, they are discussed as potential directions for future work towards deploying the proposed anomaly detection system in real-world industrial environments.

3.2.3 Real-Time Stream Ingestion and Decoupled Messaging Layer

To manage the continuous telemetry streams emitted by industrial AGVs, modern cyber-physical infrastructures decouple data ingestion from analytical processing using event-driven broker architectures. Protocols such as VDA 5050 standardize state and order exchanges via Message Queuing Telemetry Transport (MQTT) over wireless networks [37, 13]. As fleet sizes increase and robots transmit telemetry at frequent intervals, the resulting volume of concurrent messages requires scalable streaming infrastructures capable of handling high data rates, backpressure, and bursty traffic.

Distributed stream-processing engines (e.g., Apache Spark Structured Streaming) can ingest raw MQTT payloads, perform schema validation, append ingestion metadata, and cleanse malformed records in near real-time. The processed stream is then published to high-performance event brokers (e.g., Redpanda or Apache Kafka), establishing a publish-subscribe boundary that isolates edge telemetry generation from downstream machine learning consumers and storage engines.

3.2.4 The Data Lakehouse Paradigm and Medallion Storage Model

To provide structured, highly performant access to historical telemetry without incurring the costs of proprietary data warehouses, the system adopts a **Data Lakehouse** paradigm. This hybrid model superimposes DBMS-like ACID transactions, schema enforcement, and versioning over low-cost object repositories (S3-compatible storage). Data persistence is structured according to the **Medallion Architecture**, which organizes data into progressive refinement tiers [21, 22]:

1. **Bronze Layer (Raw Telemetry Ingestion):** Acts as an append-only, immutable landing zone. Incoming raw JSON payloads from the broker layer are persisted in their original format to preserve an auditable lineage of historic signals.
2. **Silver Layer (Cleaned and Enriched Data):** Stores validated, deduplicated, and schema-enforced events. Signals are converted into columnar formats (e.g., Apache Parquet) managed by open table formats like **Apache Iceberg**. This layer serves as the primary feature source for downstream deep learning models (e.g., GRU-Autoencoders).
3. **Gold Layer (Business Aggregations and Diagnostic Outputs):** Holds aggregated operational metrics, statistical summaries, and model outputs—such as reconstruction error scores and mapped diagnostic fault labels—enabling performant analytical queries.

3.2.5 Transactional Table Management and Partitioning with Apache Iceberg

In a Data Lakehouse architecture, managing growing telemetry datasets requires consistent table updates and efficient data organization. Apache Iceberg addresses these requirements through snapshot-based table management and flexible partitioning

[22].

Snapshots provide a consistent view of a table at a given point in time, allowing analytical queries to access committed data while new updates are being written. This is particularly relevant for telemetry pipelines in which data ingestion and downstream analytical processing operate concurrently.

Furthermore, Iceberg supports hidden partitioning and partition evolution, allowing tables to be organized according to relevant attributes, such as event timestamps, while adapting the partitioning strategy as data volumes and query patterns change. These capabilities can improve query efficiency and reduce the need for disruptive changes to the physical data layout [28].

3.2.6 High-Performance Time-Series Persistence and Operational Serving Layers

While Data Lakehouses provide cost-effective long-term persistence and batch analytics, operational decision-making demands sub-second latency for live dashboarding and alerting [21]. Consequently, live predictions and decomposed anomaly scores are mirrored into high-performance time-series databases (e.g., QuestDB). Optimized for column-oriented time-series indexing, QuestDB enables real-time SQL querying for operational dashboards (e.g., Grafana), completing the pipeline from continuous stream ingestion to human-interpretable predictive maintenance alerts.

4

Requirements Analysis and Specification

5

System Architecture Design

5.1 Introduction

In this chapter, we delve deeper into the system architecture design. As illustrated in Figure 5.1, the proposed solution is structured into distinct, modular tiers, forming an End-to-End Streaming Architecture.

The system relies on the following core layers:

- **Source Systems Layer:** This tier acts as the primary origin of truth. For the scope of this study, it comprises a high-performance simulation environment that generates all necessary operational and physical telemetry variables for subsequent analytics. In a prospective industrial deployment, this layer can be seamlessly extended to incorporate physical, production-grade autonomous mobile robots.
- **Broker MQTT Layer:** Serving as the initial edge gateway, this layer receives live telemetry payloads published directly by the source systems. While future iterations of the architecture might bypass this boundary—routing data directly through the central event streaming bus—deploying an independent MQTT broker establishes a robust and lightweight communication bridge tailored for IoT edge devices and complies with the requirements of the VDA5050 protocol.
- **Event Streaming Layer:** Functioning as the central nervous system of the application, this layer acts as a distributed event bus. It decouples the data emitters from the compute engines, guaranteeing reliable, low-latency message buffering and high-throughput streaming across the pipeline.
- **Stream & Processing Layer:** Dedicated to continuous stream processing, this layer performs real-time computations and data transformation. It is architected to efficiently handle and process massive volumes of high-frequency data streams.

- **Data Lakehouse Layer:** Operating as the persistence backbone, this layer stores the historical telemetry required for downstream machine learning analytics. By adopting a Data Lakehouse paradigm, the system combines cost-effective object storage with DBMS-like ACID transactions. Furthermore, it implements the *Medallion Architecture* (Bronze, Silver, and Gold tiers), preserving raw immutable ingestion data to ensure full reprocessing flexibility over time.
- **Machine Learning Layer:** This analytical module fulfills the primary objective of the research: real-time anomaly detection through advanced machine learning models consuming refined features from the data lakehouse.
- **Observability Layer:** Once operational metrics and diagnostic outputs are consolidated in the Gold layer, this tier presents actionable insights to operators through comprehensive visualization dashboards, enabling real-time monitoring of system health and robot anomalies.

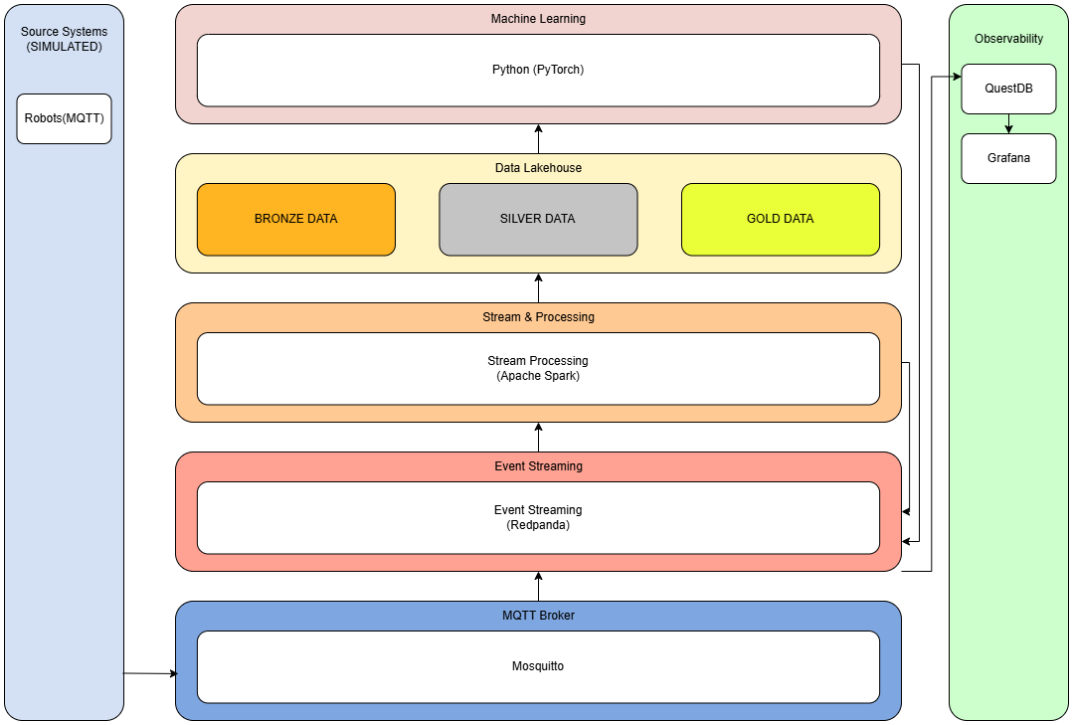


Figure 5.1. *High-level conceptual architecture of the end-to-end anomaly detection platform.*

5.2 General Architectural Decisions

Implementation: Chapter 6, Sections 6.2 and 6.3.

In addition to the component-specific design decisions presented in the following sections, several cross-cutting architectural principles were established to guide the development of the entire platform.

5.2.1 Containerized Microservice Architecture

The complete platform was designed following a containerized microservice approach. Each functional layer of the architecture is deployed as an independent Docker container, including the source systems, messaging infrastructure, stream processing

services, data storage components, and machine learning modules.

This modular design provides several advantages. First, it promotes loose coupling between system components, allowing individual services to be developed, tested, and maintained independently. Second, it simplifies system scalability, since each component can be replicated or upgraded without affecting the rest of the platform. Finally, it increases the long-term maintainability of the solution, as individual modules may be replaced by alternative implementations provided that the communication interfaces remain unchanged. Consequently, the architecture is centered on well-defined interfaces rather than on specific software implementations, facilitating future evolution of the platform.

5.2.2 Interface-Oriented Design

To maximize interoperability and future extensibility, communication between architectural layers is performed through standardized interfaces and message formats. This design principle ensures that a component can be replaced without requiring major modifications in the remaining layers. For example, the simulated AGV fleet used during development could be replaced by a real fleet management system, while preserving the same MQTT and VDA 5050 communication contracts. As a result, the architecture remains independent of specific hardware vendors or software providers.

5.2.3 Event-Driven Architecture

An event-driven architecture was adopted as the communication paradigm for the entire platform. Rather than relying on direct point-to-point interactions between services, all components communicate through an intermediate event streaming layer. This approach significantly reduces coupling between producers and consumers, allowing individual services to evolve independently without affecting other parts of the system. Additionally, new services can be integrated into the platform with minimal effort by subscribing to the existing event streams. The event streaming layer acts as the system's central source of truth, providing a unified and consistent mechanism for distributing telemetry, operational events, and analytical results across the architecture. This design improves scalability, extensibility, and maintainability while facilitating the integration of future data processing and monitoring capabilities.

5.2.4 Separation of Training and Inference Pipelines

The machine learning subsystem was designed following a clear separation between training and inference workloads. Model training is performed offline using Python-based machine learning frameworks that consume historical data and engineered features stored in Apache Iceberg tables. This approach provides flexibility for feature engineering, experimentation, hyperparameter tuning, and model evaluation.

In contrast, model inference is integrated directly into the Apache Spark streaming pipeline. Once a model has been trained and validated, it is exported and embedded into the stream processing layer, allowing predictions to be generated in real time as new AGV telemetry events are processed. This design avoids the latency associated with external prediction services and enables seamless integration with the event-driven architecture.

The separation of training and inference responsibilities allows each workload to be optimized independently. Training processes prioritize model accuracy and experimentation capabilities, whereas inference processes prioritize low latency, scalability, and operational reliability. Furthermore, the architecture facilitates future model updates, since newly trained models

can be deployed to the streaming environment without requiring modifications to the surrounding data processing infrastructure.

5.3 Source Systems Layer

Implementation: Chapter 6, Section 6.4.

In designing the data ingestion pipeline, a preliminary study was conducted to determine the optimal source of truth for the system. At first, the possibility of utilizing a physical Autonomous Mobile Robot (AMR) in real time was evaluated. However, this approach was discarded for the training phase due to two primary constraints: first, only a single real unit was available, which is insufficient to represent a fleet-scale source system; second, the physical hardware lacked the specific internal telemetry sensors required for comprehensive anomaly detection analytics.

As a second approach, a search was conducted for publicly available industrial datasets containing multi-variable telemetry from AGVs and AMRs. While several sources were identified, none provided the precise combination of electrical, mechanical, and thermal variables necessary for the anomaly detection module.

Given the limitations of both physical deployments and existing public datasets, the decision was made to implement a high-performance simulation framework capable of emulating an entire fleet of AGVs deterministically. Since the chosen simulation environment, built upon the open-source repository `bekirbostanci/vda5050_vehicle_simulator` [5], did not natively incorporate deep physical powertrain dynamics, it was adapted to model core electromechanical and thermal variables. To ensure seamless integration with downstream data consumers, the messages published by the simulated robots strictly adhere to the **VDA 5050** [37] interoperability standard, extended to include custom telemetry payloads.

5.4 MQTT Broker Layer

Implementation: Chapter 6, Section 6.5.

The adoption of the VDA 5050 communication standard directly influenced the design of the messaging layer. Since VDA 5050 defines MQTT as the underlying communication protocol between fleet management systems and AGVs, the incorporation of an MQTT broker became a fundamental architectural requirement.

The broker layer acts as an intermediary between telemetry producers and data consumers, enabling asynchronous communication and decoupling the different system components. This approach allows producers and consumers to evolve independently while maintaining interoperability through standardized message exchanges.

Several MQTT broker implementations were evaluated during the architectural design phase. Eclipse Mosquitto was ultimately selected due to its maturity, open-source licensing model, low resource consumption, Docker compatibility, and widespread adoption within Industrial Internet of Things (IIoT) environments.

Furthermore, the publish/subscribe communication model facilitates horizontal scalability and simplifies the integration of future services. New consumers can subscribe to existing telemetry streams without requiring modifications to the source systems, making the architecture more extensible and maintainable.

5.5 Event Streaming Layer

Implementation: Chapter 6, Section 6.6.

To establish an enterprise-grade backbone for telemetry ingestion, storage, and analytics, the architecture incorporates a distributed event streaming platform. While the MQTT layer excels at lightweight device-to-broker messaging, an append-only distributed log is essential to serve as the immutable single source of truth, handle high-throughput event replay, and decouple analytical consumers from edge producers.

5.5.1 Kafka provider selection

During the architectural design phase, distributed streaming technologies were evaluated. Although Apache Kafka represents the industry benchmark, **Redpanda** was selected as the central event broker. Redpanda provides complete protocol-level compatibility with the Apache Kafka API, allowing ecosystem tools—such as Apache Spark Structured Streaming and Kafka Connect—to operate seamlessly without modification or proprietary client adapters.

The decision to adopt Redpanda instead of traditional Apache Kafka is justified by several architectural advantages:

- **Native Performance and Predictable Latency:** Redpanda is engineered in C++ utilizing the Seastar asynchronous framework with a thread-per-core architecture. This eliminates the Java Virtual Machine (JVM) runtime overhead and avoids Stop-the-World Garbage Collection (GC) pauses.
- **Architectural Simplicity and Self-Contained Topology:** Unlike traditional Kafka deployments requiring dedicated Apache ZooKeeper clusters or resource-intensive KRaft JVM controllers, Redpanda implements native Raft[23] consensus embedded directly within the broker binary. This significantly reduces operational complexity, container footprint, and deployment startup times.
- **Resource Efficiency in Edge and Development Environments:** The lightweight footprint of Redpanda enables high throughput and reliable partitioned persistence while conserving host computing resources, making it exceptionally well-suited for containerized edge gateways and Proof-of-Concept (PoC) deployments.
- **Integrated Developer Tooling:** The platform bundles the `rpk` management utility and the Redpanda Console user interface out of the box, streamlining topic inspection, consumer-group monitoring, and partition offset observability without requiring external monitoring containers.

By leveraging Redpanda as the event streaming backbone, the platform ensures durable event persistence, horizontal scalability, ordered event processing, and seamless integration with the wider Kafka ecosystem. These characteristics make it an appropriate foundation for the real-time data ingestion and analytics requirements of the proposed architecture.

5.5.2 Kafka Topic Design

The creation of topics in Redpanda follows specific design considerations to efficiently organize telemetry streams, operational events, and analytical data flows. Topics were designed to provide clear separation of concerns, facilitate independent consumer processing, and support the event-driven communication model adopted throughout the platform. Table 5.1 summarizes the topic naming conventions, data content, and information flow within the proposed architecture.

A defining element of the topic design is the selection of a consistent partitioning key across the streaming backbone. All telemetry topics use the composite key `manufacturer : serialNumber`, which in the VDA 5050 standard unambiguously

Table 5.1. *Kafka topic design, data flow, and partitioning key.*

Topic name	Partition key	Name selection	Content	Produced by	Consumed by
`vda5050-state`	`manufacturer serialNumber`	Reflects the standardized VDA 5050 state messages generated by automated guided vehicles (AGVs) [37].	Raw telemetry and operational state messages contained in the VDA 5050 payload.	MQTT broker, bridged through Telegraf.	Apache Spark Structured Streaming.
`vda5050-state-deduplicated`	`manufacturer serialNumber`	Identifies cleaned state messages, distinguishing genuine state transitions from redundant heartbeat updates.	Deduplicated VDA 5050 state records preserving genuine physical and operational changes.	Apache Spark Structured Streaming.	Apache Spark Structured Streaming.
`fleet-features`	`manufacturer serialNumber`	Represents fleet-level operational features extracted and enriched from raw vehicle telemetry.	Processed telemetry, including derived kinematic and electrical features.	Apache Spark Structured Streaming.	Apache Spark Structured Streaming and QuestDB via Kafka Connect.
`ai-anomalies`	`manufacturer serialNumber`	Identifies the outputs of the artificial intelligence module, distinguishing anomaly results from raw telemetry and derived features.	Inference outputs, including anomaly scores, reconstruction errors, and diagnostic information.	Apache Spark Structured Streaming.	QuestDB via Kafka Connect.

identifies a single physical vehicle [37]. This choice is motivated by two complementary architectural guarantees. First, because a Kafka-compatible log preserves ordering only *within* a partition, routing every message of a given AGV to a fixed partition ensures strict chronological ordering of its state transitions, a prerequisite for correct kinematic reconstruction and windowed feature computation downstream. Second, prefixing the serial number with the manufacturer prevents hash collisions between distinct vehicles that could otherwise share an identical serial number. At the same time, different vehicles are hashed to different partitions, allowing analytical consumers such as Apache Spark to process distinct AGVs in parallel. The key is therefore applied uniformly across the ingestion, feature, and inference topics so that the per-asset ordering guarantee is preserved end to end across the Bronze, Silver, and Gold tiers.

5.6 Streaming Integration and Inter-Service Connectivity

Implementation: Chapter 6, Section 6.7.

A critical requirement of the event-driven architecture is ensuring clean separation of concerns across the ingestion, streaming, storage, and processing tiers. Rather than tightly coupling producers and consumers to the central streaming bus, the integration topology relies on standardized connectors and specialized bridges. This approach isolates protocol adaptations to the respective system boundaries, preserving the raw VDA 5050 telemetry format within the central event log, more details in the following subsections.

The end-to-end data pipeline and inter-service connectivity model are illustrated in Figure 5.2.

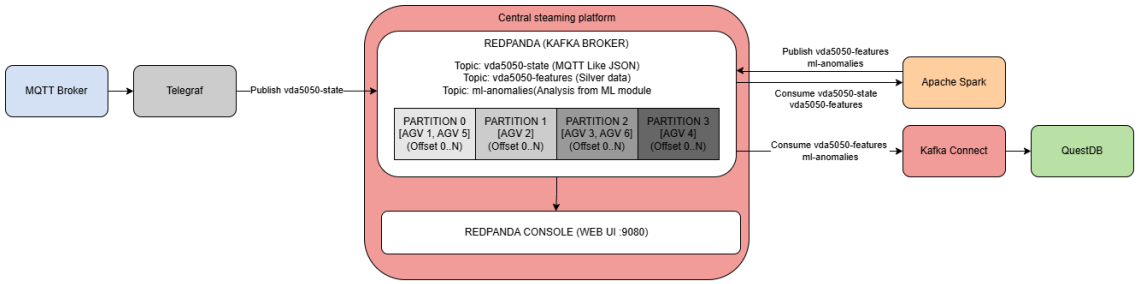


Figure 5.2. *End-to-end integration topology connecting MQTT source systems, Redpanda streaming bus, QuestDB time-series storage via Kafka Connect, and Apache Spark analytics.*

The rationale and implementation details for each integration interface are summarized below:

5.6.1 MQTT to Event Streaming Ingestion Layer

An architectural decision was required to establish the data bridge between the edge messaging layer (Eclipse Mosquitto) and the central event streaming bus (Redpanda), ensuring that vehicle telemetry producers remain decoupled from Kafka-specific protocols.

During the design phase, deploying a dedicated Apache Kafka Connect MQTT Source connector was evaluated. However, the MQTT ingestion boundary only requires protocol adaptation between MQTT and Kafka-compatible event streams, without the need for advanced connector lifecycle management, schema evolution, or stream transformations. Consequently, introducing a full Kafka Connect framework at this stage was considered architecturally unnecessary. Instead, Telegraf was selected as

a dedicated telemetry ingestion agent because it provides native MQTT consumer and Kafka producer capabilities, enabling direct protocol bridging while preserving the original telemetry payloads. This approach minimizes architectural complexity, reduces the number of operational dependencies, and keeps the ingestion boundary focused exclusively on message transport responsibilities.

5.6.2 Streaming to Time-Series Persistence: Decoupled Kafka Connect for QuestDB

For real-time operational monitoring and visualization in Grafana, events must be streamed continuously into **QuestDB**. Instead of relying on bespoke consumers or embedding message-bus client logic into the storage layer, the industry-standard **Apache Kafka Connect** framework was adopted in sink mode (`io.questdb.kafka.QuestDBSinkConnector`).

NEEDS TO BE COMPLETED!

5.6.3 Event Streaming to Analytical Processing Connectivity

An architectural evaluation was conducted to determine how to interface the central event streaming bus (Redpanda) with the analytical engine (**Apache Spark**) for downstream anomaly detection workloads.

Specifically, it was assessed whether an intermediate connector service or an external data-staging layer was required to bridge both systems. It was decided to connect Apache Spark directly to Redpanda by leveraging Spark's built-in, native Kafka streaming support.

Because Apache Spark incorporates first-class Kafka protocol consumer capabilities out of the box, deploying a dedicated connector service or third-party middleware was deemed unnecessary. This decision significantly simplifies the deployment topology, reduces the overall component footprint, and eliminates architectural redundancy, allowing analytical workloads to consume raw telemetry streams directly from the message bus with minimal latency.

5.7 Stream & Processing Layer: Apache Spark

Implementation: Chapter 6, Section 6.8.

In defining the computational backbone for telemetry transformation, analytical feature engineering, and scalable inference, an architectural decision was made regarding the stream processing engine. **Apache Spark**, utilizing its **Structured Streaming** framework, was selected as the central engine for this layer.

In accordance with the scope of this chapter, this section outlines the architectural design decisions, component responsibilities, and data contracts that govern the stream processing tier, whereas the implementation is addressed in (Chapter 6).

While distributed event logs like Redpanda provide durable message buffering, they lack high-level compute primitives for complex multi-stage data processing and distributed machine learning inference. Apache Spark was selected based on the following architectural design principles:

- **End-to-End Horizontal Scalability:** Telemetry volumes scale proportionally with the number of operational AGVs. Decoupling stream processing into a distributed engine ensures that computational bottlenecks are eliminated at the architectural level, allowing processing capacity to scale across partitions without altering application logic.

- **First-Class Lakehouse Integration:** Selecting an engine with native support for open table formats (**Apache Iceberg**) ensures transactional consistency (ACID guarantees), snapshot isolation, and seamless interoperability with the S3-compatible storage layer without requiring external data ingestion services.
- **Adoption of a Dual-Sink Streaming Pattern:** Rather than coupling stream processing to a single egress target, it was decided to adopt a dual-sink architectural pattern across processing stages. Under this design decision, Spark concurrently commits processed records to historical Iceberg tables for long-term analytics while republishing the transformed events back into dedicated Redpanda topics for low-latency downstream consumption.

From an architectural perspective, the trade-off between continuous, single-event streaming (as offered by engines such as Apache Flink or Kafka Streams) and the micro-batch execution model was evaluated. While single-event engines achieve marginally lower nominal latencies, the micro-batch approach was selected because the sub-second latency provided by Spark Structured Streaming fully satisfies the temporal requirements of AGV telemetry (sampling between 1 Hz and 10 Hz). Furthermore, micro-batching significantly simplifies transaction coordination, commit atomicity, and file compaction when interfacing with lakehouse table formats, offering a more balanced architectural trade-off for this platform.

5.7.1 Medallion Processing Pipeline and Job Topology

To avoid monolithic processing architectures and enforce strict separation of concerns across data maturity levels, the stream processing tier is decomposed into three decoupled streaming stages following the **Medallion Architecture** pattern (Bronze, Silver, and Gold tiers). Each stage is architecturally designed around distinct operational contracts, input/output interfaces, and analytical responsibilities:

1. **Job 1: VDA5050 State Writer (Raw to Bronze Persistence)** The telemetry generated by AGVs constitutes a continuous stream of operational data. While this stream is suitable for real-time communication between system components, it cannot be considered a reliable long-term source for analytics, auditing, or machine learning workloads. Messages stored only in Kafka are inherently transient and depend on retention policies, making them unsuitable as the primary source for historical analysis. Consequently, a persistent storage layer is required to preserve the complete history of vehicle telemetry. This historical repository enables offline data exploration, reproducible machine learning experiments, anomaly investigations, and system traceability. Several architectural alternatives were considered during the design phase. One possibility consisted of storing only the latest state of each AGV. This approach was discarded because anomaly detection models require access to historical behaviour patterns rather than individual snapshots. Another alternative involved applying state-change deduplication during ingestion to reduce storage requirements. However, due to the continuous variation of sensor readings and the stochastic noise introduced by the simulator, consecutive messages rarely contain identical payloads. The storage savings obtained from deduplication would therefore be limited while introducing additional processing complexity. For these reasons, the Bronze layer was designed as an immutable repository containing all incoming VDA5050 telemetry messages without modifications, filtering operations, or feature extraction. The `vda5050-state-writer` job is responsible for materialising this architectural decision by persisting the complete telemetry stream from the `vda5050-state` topic into an Apache Iceberg table. The resulting dataset serves as the authoritative historical record of the system.

2. **Job 2: Bronze to Silver (Feature Extraction and Kinematic Aggregations)** Although the Bronze layer preserves the original telemetry stream, the VDA5050 message format is primarily designed for interoperability between fleet management systems and AGVs rather than analytical processing. As a consequence, messages contain nested structures and arrays that introduce unnecessary complexity for querying, statistical analysis, and machine learning workflows. To decouple the operational representation of telemetry from analytical requirements, a dedicated Silver layer was introduced. The objective of this layer is to expose a clean, structured, and domain-oriented representation of the information that can be consumed consistently by data analytics and machine learning components. The transformations performed in this stage are described below.

Flattening

The first design decision concerns the representation of telemetry data. Directly exposing nested VDA5050 messages to downstream consumers would require every analytical component to repeatedly navigate hierarchical structures. Such an approach would increase implementation complexity and reduce interoperability between processing modules. To address this issue, a flattening process converts the original hierarchical message into a tabular schema. This representation is more suitable for Apache Spark processing, Apache Iceberg storage, and future machine learning pipelines while preserving the semantic meaning of the original telemetry data.

Feature Selection

A second design objective consists of identifying which information is genuinely relevant for anomaly detection. The original VDA5050 messages contain numerous operational fields related to order execution, navigation state, and protocol management. However, the anomaly detection use case considered in this work focuses on identifying abnormal physical behaviour of AGVs through their telemetry measurements. For this reason, the selected features prioritise vehicle kinematics, battery indicators, motor telemetry, and safety-related signals while excluding protocol-oriented information that does not contribute to the detection of physical anomalies. The selected Silver-layer schema is composed of the following attributes:

- `timestamp`: Event timestamp used for temporal ordering and sequence reconstruction.
- `manufacturer`: AGV manufacturer identifier.
- `agv_id`: Unique AGV identifier.
- `map_id`: Identifier of the operational environment.
- `pos_x`, `pos_y`: Vehicle position coordinates.
- `vx`, `vy`, `omega`: Linear and angular velocity components.
- `driving`: Vehicle motion state.
- `battery_charge`: Remaining battery charge level.

- `battery_health`: Battery health indicator.
- `battery_voltage`: Battery voltage measurement.
- `battery_current`: Battery current measurement.
- `battery_temperature`: Battery temperature measurement.
- `charging`: Battery charging state.
- `motor_current`: Motor current consumption.
- `motor_voltage`: Motor voltage measurement.
- `motor_temperature`: Motor temperature measurement.
- `motor_speed_rad_s`: Motor rotational speed.
- `motor_torque_nm`: Motor torque.
- `motor_power_w`: Motor power consumption.
- `wheel_slip`: Wheel slip ratio.
- `back_emf_v`: Back electromotive force voltage.
- `fault_active`: Active fault indicator.

The resulting Silver-layer schema establishes a common analytical representation of AGV telemetry data. This dataset serves as the foundation for subsequent machine learning processes, providing a consistent and domain-oriented view of the physical state of the vehicles while remaining independent of the original VDA5050 message structure.

3. **Job 3: Silver to Gold (Distributed Real-Time Anomaly Inference):** TO DO!

Figure 5.3 illustrates the end-to-end data flow across the three Spark streaming jobs, detailing the interaction with Redpanda topics and S3 Iceberg tables.

5.8 Data Lakehouse Layer: Storage Architecture

Implementation: Chapter 6, Section 6.9.

To establish the analytical persistence foundation of the platform, the architecture incorporates the **Data Lakehouse** paradigm. While the event streaming bus (Redpanda) provides transient message buffering and the time-series engine (QuestDB) serves real-time dashboards, an immutable, cost-effective storage tier is required to persist the multi-tier Medallion datasets (Bronze, Silver, and Gold), store streaming execution checkpoints, and serve historical telemetry for offline machine learning model training.

5.8.1 Object Storage Provider Selection: MinIO

In modern data architectures, decoupling compute from storage is a fundamental design principle. Rather than deploying traditional distributed file systems (such as Apache HDFS) that introduce heavy operational overhead and tight coupling between compute nodes and physical disks, the architecture adopts **Object Storage** as the core persistence medium.

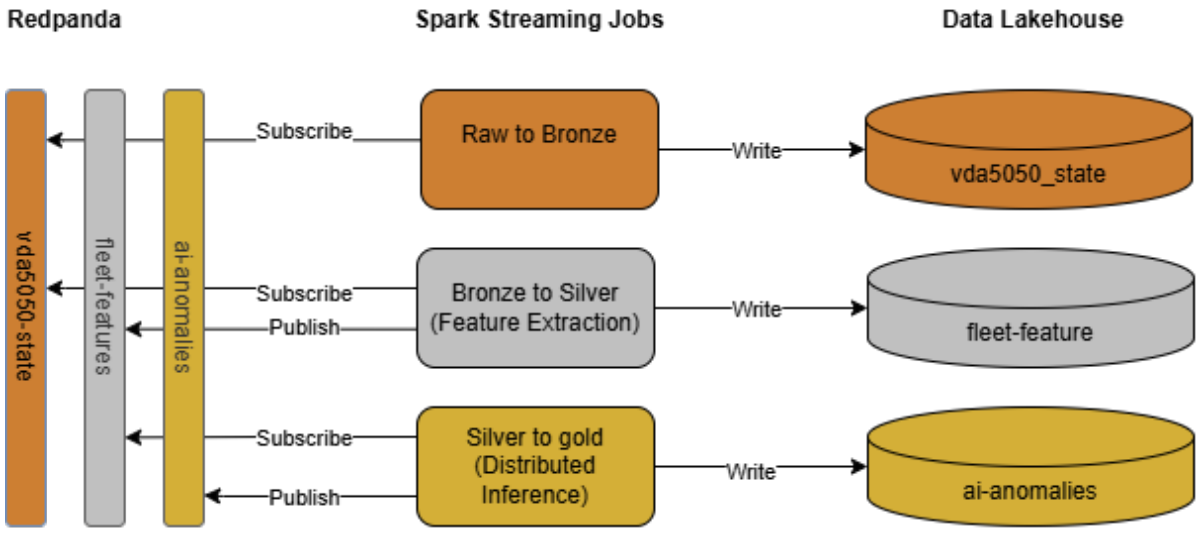


Figure 5.3. *Data pipeline of the multi-stage Spark Structured Streaming architecture, showing dual-sink persistence to Redpanda topics and S3 Iceberg tables.*

During the architectural evaluation of object storage engines for on-premise, edge, and containerized deployment, **MinIO** was selected based on the following architectural criteria:

- **Industry-Standard S3 API Compatibility:** MinIO provides complete protocol-level compatibility with the Amazon S3 REST API. This native compatibility allows upstream compute engines—specifically Apache Spark via its `hadoop-aws` S3A connector—and modern table management engines (Apache Iceberg) to interface with the storage tier using standard S3 client libraries without requiring proprietary adapters.
- **Hybrid and Edge Deployment Portability:** In industrial cyber-physical environments, telemetry architectures frequently run on local edge infrastructure or on-premise factory servers before synchronizing with corporate data centers. MinIO enables a cloud-native S3 storage layer directly within containerized on-premise environments, ensuring that analytical pipelines can be migrated seamlessly to public cloud object stores (such as AWS S3 or Google Cloud Storage) simply by updating connection endpoints, without modifying data access logic.
- **High-Throughput Columnar and Micro-Batch Workloads:** Written in Go, MinIO delivers lightweight resource consumption and high sequential read/write throughput. These characteristics are critical for handling the concurrent demands of Apache Spark's streaming checkpoints and continuous Parquet file ingestion.
- **Strong Consistency for Lakehouse Table Formats:** Modern lakehouse table formats, such as Apache Iceberg, rely on atomic commit guarantees and strong read-after-write consistency to manage metadata snapshots and prevent race conditions during concurrent write operations. MinIO provides immediate consistency for all object operations out of the box, fulfilling this foundational requirement.

5.8.2 Table Format Selection: Apache Iceberg

Object storage engines such as MinIO provide durable byte-level persistence, but relying directly on raw columnar files (for example, bare Apache Parquet) introduces significant limitations in a concurrent streaming environment. In particular, raw files offer no transactional guarantees, suffer from the "small files problem" under continuous micro-batch appends, and can expose dirty or partial reads when an analytical query overlaps with an ongoing write. To overcome these issues, modern Lakehouse architectures interpose an open **Table Format** layer between the object store and the compute engines, endowing the data lake with database-grade semantics [15].

The three leading open table formats—**Apache Hudi**, **Delta Lake**, and **Apache Iceberg**—were considered during the design phase. **Apache Iceberg** was selected as the analytical table format of the platform on the basis of the following architectural drivers [22]:

- **Engine-Agnostic, Vendor-Neutral Specification:** Unlike Delta Lake, which has historically been closely associated with the Spark and Databricks ecosystem, Iceberg was designed from its inception as an open, engine-neutral specification. This enables the write path to rely on Spark Structured Streaming while leaving the read path free to use distributed SQL engines, DuckDB, or pure Python tooling (for example, `pyiceberg`) for offline machine learning, without runtime lock-in.
- **ACID Transactions and Snapshot Isolation:** Iceberg provides serializable isolation through an optimistic concurrency model built on atomic metadata commits. This guarantees that continuous micro-batch ingestion performed by Spark never interferes with the long-running, read-heavy scans issued by offline model-training routines, as readers always observe a consistent table snapshot.
- **Fit for Append-Only Telemetry Workloads:** While Apache Hudi is heavily optimized for Change Data Capture scenarios with frequent record-level updates and deletes, cyber-physical AGV telemetry is strictly append-only. Iceberg offers a leaner metadata layout better suited to high-throughput sequential appends and automated file compaction, without the record-indexing overhead that Hudi introduces.
- **Safe Schema Evolution:** As robotic platforms evolve, their telemetry payloads may add new diagnostic fields or retire obsolete ones. Iceberg supports column addition, renaming, and reordering as a purely metadata-level operation, preserving backward compatibility without rewriting historical Parquet files.

5.8.3 Automated Lakehouse Maintenance Architecture

Implementation: Chapter 6, Section 6.8.6.

While low-latency micro-batching (15 seconds for Bronze and 3 seconds for Silver) fulfills the near-real-time ingestion requirements of the platform, it inherently introduces the *small files problem* as an architectural side effect. Over continuous 24/7 industrial operation, appending new Parquet data files every few seconds produces tens of thousands of fragmented objects and inflates the commit snapshot tree within object storage. Left unmitigated, this file fragmentation severely degrades the metadata listing speed, partition pruning efficiency, and sequential scanning throughput of downstream machine learning training workflows.

To prevent performance degradation without burdening the real-time ingestion path, the architecture incorporates an independent, decoupled **Lakehouse Maintenance Tier**. Rather than performing inline compaction during micro-batch execution—which would destabilize streaming latencies—maintenance is orchestrated as an auxiliary operational component operating outside the main Medallion data flow. Architecturally, the maintenance tier addresses three distinct optimization concerns:

1. **Data File Compaction:** Merging thousands of small micro-batch Parquet files into standardized target-sized columnar blocks (64 MiB) to optimize sequential I/O and columnar compression ratios.
2. **Manifest Optimization:** Re-indexing and defragmenting Iceberg manifest files and manifest lists to accelerate file pruning during query planning.
3. **Snapshot and Metadata Expiration:** Purging historical snapshot metadata and unreferenced orphaned Parquet data files that have been superseded by compacted files, bounding storage growth and metadata parsing overhead.

Crucially, the maintenance process is triggered on demand through automated metadata inspection rather than arbitrary execution schedules, optimizing compute resource utilization in resource-constrained edge deployments.

5.8.4 Bifurcated Storage Architecture: Operational vs. Analytical Persistence

A core architectural decision of the proposed platform is the explicit decoupling between the **operational time-series store** (QuestDB) and the **analytical object store** (MinIO):

- **Operational Path (QuestDB):** Optimized for low-latency ingest and high-frequency SQL aggregation across rolling time windows, dedicated to live operator visualization in Grafana.
- **Analytical Path (MinIO / Lakehouse):** Optimized for massive, append-only historical telemetry persistence in columnar Parquet format, multi-tier Medallion curation, and high-throughput batch scanning required for offline machine learning model training.

This bifurcated storage design isolates analytical queries and heavy machine learning workloads from live operational monitoring, ensuring that intensive model training scans do not degrade dashboard rendering performance or message ingestion throughput.

5.9 Machine Learning layer

Implementation: Chapter 6, Section 6.11. TO DO!

5.10 Observability Layer: Grafana Dashboards

Implementation: Chapter 6, Section 6.12. TO DO!

5.11 Security and Access Control Considerations

Implementation: Chapter 6, Section 6.13.

6

System Implementation

6.1 Introduction

As discussed in Chapter 5, the proposed solution follows an end-to-end streaming architecture composed of multiple interconnected layers.

While the previous chapter focused on the architectural design decisions and the rationale behind the selected technologies, this chapter presents the practical implementation of the system. First, Section 6.2 introduces the implementation methodology, version control practices, and continuous integration workflows. Subsequently, the implementation details of each architectural layer are described, together with the deployment strategy, service integration mechanisms, and the technologies used to establish communication between components.

Finally, the chapter explains how the different layers are combined to form a complete data pipeline, from telemetry generation at the source systems to data storage and downstream analytics.

6.2 Implementation Methodology

Architectural counterpart: Chapter 5, Section 5.2.

Translating the high-level architectural design presented in Chapter 5 into an operational platform requires establishing a structured implementation methodology. The proposed solution constitutes a distributed and heterogeneous system that spans physical vehicle simulation engines written in Rust, IoT edge messaging brokers, distributed stream processors powered by Apache Spark, cloud-native lakehouse storage tiers, and deep learning models built in PyTorch. Managing such a multi-technology platform demands disciplined software engineering and DevOps practices to guarantee reproducibility, traceability, and seamless interoperability between components.

6.2.1 Distributed Version Control and Commit Traceability

Modern software engineering relies on distributed version control to record changes, maintain history, and coordinate technical iterations. In this project, Git [14] was selected as the central version control system, hosted within the Bitbucket [2] repository infrastructure of the industrial partner.

6.2.2 Continuous Integration and Automated Verification

Continuous Integration (CI) and Continuous Deployment (CD) principles were integrated into the development workflow to ensure that modifications to individual microservices do not destabilize the overall pipeline. The containerized design of the platform directly supports automated verification workflows:

- **Automated Build Validation:** Because system components are containerized, automated build steps verify that Docker images can be built without errors, ensuring that third-party dependencies and compilation targets (such as the Rust-based vehicle simulator) remain consistent and pinned.
- **Quality Assurance and Static Analysis:** Static code analysis, syntax checking, and formatting tools were applied across the various languages utilized in the project (Python, Rust, shell scripts, and configuration files) to enforce consistent code quality standards.
- MORE WILL BE ADDED!

6.3 Containerized Deployment

Architectural counterpart: *Chapter 5, Section 5.2.* Following the architectural decisions described in Chapter 5, all components were deployed as Docker containers. Containerization provides reproducibility, isolation of dependencies, and simplifies the deployment and maintenance of the platform.

6.3.1 Docker Compose Organization

To improve maintainability and modularity, the platform is organized as a collection of independent Docker Compose files. Each architectural layer maintains its own deployment definition, while a main `docker-compose.yml` file aggregates all services using the Docker Compose `include` directive.

This approach allows each subsystem to be developed and deployed independently while preserving a unified deployment strategy for the complete platform.

Listing 6.1 shows the complete root Docker Compose configuration used to orchestrate all subsystems, define CA certificates, and declare shared network boundaries.

```
secrets:
  ca_cert:
    file: ./certs/edag-r1.crt

include:
  - docker/mqtt-broker/docker-compose.yml
  - docker/source_systems/vda5050_vehicle_simulator/docker-compose.yml
  - docker/redpanda/docker-compose.yml
  - docker/source_systems/telegraf/docker-compose.yml
  - docker/data-storage/questdb/docker-compose.yml
```

```
  - docker/cross-cutting/docker-compose.yml
  - docker/utils/sim-control-panel/docker-compose.yml
  - docker/stream-processing/spark/docker-compose.yml
  - docker/data-lakehouse/minio/docker-compose.yml

networks:
  lakehouse-net:
    driver: bridge
  redpanda-net:
    driver: bridge
  mqtt-net:
    driver: bridge
  questdb-net:
    driver: bridge
```

Listing 6.1. *Root Docker Compose configuration*

6.3.2 Network Segmentation

Dedicated Docker bridge networks were created to isolate communication between system components. Rather than exposing every service to a common network, individual networks were assigned according to the communication requirements of each layer.

This strategy reduces unnecessary connectivity between containers and simplifies service discovery. Furthermore, it follows the principle of least privilege by restricting access only to the services required by each component.

The platform defines four segmented bridge networks:

- `mqtt-net`: Dedicated to edge communication between simulated AGVs, the Eclipse Mosquitto broker, and the Telegraf ingestion agent.
- `redpanda-net`: Connects the central distributed event streaming brokers, Telegraf producer, Spark streaming consumers, and Kafka Connect.
- `lakehouse-net`: Dedicated to the analytical storage tier, connecting Spark stream processing with the MinIO S3 object store.
- `questdb-net`: Isolates the operational time-series database, schema initialization containers, Kafka Connect sink, and Grafana monitoring dashboards.

6.3.3 Modular Service Configuration

To improve modularity, maintainability and deployment portability, all infrastructure dependencies are configured externally through environment variables. Rather than hardcoding service endpoints, credentials or storage locations within the application

source code, each component retrieves its configuration at runtime from the shared `.env` file. This design allows individual services to be replaced without requiring modifications to the application logic. As a consequence, Spark jobs, Kafka clients, Iceberg catalogs and storage backends remain loosely coupled and can be adapted to different deployment environments through configuration only. This enables the same container images to be reused across development, testing and production deployments. For example, the current implementation uses Redpanda as the streaming platform:

```
KAFKA_BROKER=redpanda:9092
```

If the system were migrated to a different Kafka-compatible broker, only the endpoint definition would need to be updated:

```
KAFKA_BROKER=my-broker.example.com:9092
```

No changes to the Spark jobs, data processing logic or container images would be required.

6.4 Source Systems: Vehicle Simulation and Physical Modeling

Architectural counterpart: Chapter 5, Section 5.3. This section describes the implementation of the source systems layer, the origin of truth of the platform. It covers the containerized deployment of the extended vehicle simulator, the electromechanical, kinematic, thermal and battery model that governs the generated telemetry, the physical fault-injection and transducer models, and the resulting telemetry channels published as VDA 5050 payloads.

6.4.1 Simulator Architecture and Containerized Deployment

The source system layer is currently implemented using a modified version of the `vda5050_vehicle_simulator` [5]. The simulator executes inside a dedicated container and connects to the MQTT broker through the shared `mqtt-net` network.

The `depends_on` directive combined with Docker health checks guarantees that the simulator starts only after the broker has become available.

```
services:
  simulator:
    image: vda5050-vehicle-simulator:latest
    volumes:
      - ./config.toml:/app/config.toml:ro
    networks:
      - mqtt-net
    depends_on:
      mosquitto:
        condition: service_healthy
    restart: unless-stopped
```

Listing 6.2. *VDA5050 Simulator Docker Compose*

The simulator configuration is loaded from a dedicated configuration file that defines fleet parameters, MQTT communication settings, and telemetry generation properties. During execution, each simulated AGV periodically publishes VDA 5050-compliant state messages to the MQTT broker.

To support the anomaly detection use case, the original simulator was extended with additional telemetry channels representing motor, battery, and drivetrain behavior. These extensions enable the generation of realistic training data for machine learning workloads while preserving compatibility with the VDA 5050 communication standard.

To provide physically consistent telemetry for anomaly recognition, the kinematic simulation of the automated guided vehicle (AGV) is extended with an electromechanical powertrain model. The objective is not to replicate every microscopic effect of a physical robot, but to capture the dominant coupling between trajectory demands, mechanical loads, electrical responses, and thermal dissipation.

6.4.2 Motor and Drivetrain Dynamics

The traction drive is modeled as an equivalent permanent-magnet DC (PMDC) machine coupled to a reduction gearbox (i_g) and driven wheel (r_w).

The electrical armature circuit is governed by:

$$V_m(t) = R_a(T_m)I_m(t) + L_a\frac{dI_m(t)}{dt} + K_e\omega_m(t) \tag{6.1}$$

where V_m is the armature terminal voltage, I_m the armature current, $R_a(T_m)$ the winding resistance, K_e the back-EMF constant, and ω_m the rotor speed.

Quasi-static approximation ($L_a \approx 0$):

Because the electrical time constant ($\tau_e = L_a/R_a \approx 40\,\text{ms}$) is much shorter than the mechanical time constant ($\tau_m \approx 1\text{–}3\,\text{s}$) and the state reporting rate, the inductive transient $L_a\frac{dI_m}{dt}$ is neglected. This yields the quasi-static relationship:

$$V_m(t) = R_a(T_m)I_m(t) + K_e\omega_m(t) \tag{6.2}$$

Electromagnetic torque follows:

$$\tau_m(t) = K_tI_m(t) \tag{6.3}$$

where K_t is the torque constant. The mechanical torque balance at the motor shaft is:

$$J_m\frac{d\omega_m(t)}{dt} + B_m\omega_m(t) = \tau_m(t) - \frac{\tau_L(t)}{\eta_g i_g} - \tau_{\text{friction}} \tag{6.4}$$

where J_m is rotor inertia, B_m viscous friction, η_g gearbox efficiency, and τ_{friction} unmodeled or degraded drivetrain friction. The wheel load torque $\tau_L(t)$ aggregates rolling resistance, road grade, vehicle acceleration, and drawbar forces:

$$\tau_L(t) = r_w\left[C_{rr}m_{\text{total}}g\cos\theta + m_{\text{total}}a(t) + m_{\text{total}}g\sin\theta + F_{\text{drawbar}}\right] \tag{6.5}$$

where $m_{\text{total}} = m_{\text{chassis}} + m_{\text{payload}}$ is the total vehicle mass.

6.4.3 Kinematics and Wheel Slip

The relationship between rotor speed and linear ground speed $v(t)$ accounts for transmission geometry and wheel slip:

$$v(t) = \frac{\omega_m(t)}{i_g}r_w(1 - s(t)) \tag{6.6}$$

where $s(t) \in [0, 1)$ represents the slip ratio. Under nominal traction, $s(t) \approx 0$. When wheel slippage occurs, $s(t) \to 1$, which decouples wheel speed from chassis velocity ($v_{\text{wheel}} \gg v_{\text{ground}}$) while collapsing the ground reaction torque ($\tau_L \to 0$).

6.4.4 Thermal Dissipation Models

Winding resistance increases with copper temperature:

$$R_a(T_m) = R_{a0}\left[1 + \alpha(T_m(t) - T_0)\right] \tag{6.7}$$

where R_{a0} is the baseline resistance at temperature T_0, and α is the copper temperature coefficient.

The thermal dynamics of the motor are modeled as a lumped first-order node accounting for Joule heating, core iron losses (Steinmetz formulation), and convective heat transfer to the ambient environment (T_{amb}):

$$C_{\text{th}}\frac{dT_m(t)}{dt} = I_m(t)^2 R_a(T_m) + C_{fe}\omega_m(t)^{1.6}B_{\text{max}}^2 - \frac{T_m(t) - T_{\text{amb}}}{R_{\text{th}}} \tag{6.8}$$

Similarly, the battery pack temperature $T_{\text{bat}}(t)$ accounts for internal resistive heating:

$$C_{\text{th,bat}}\frac{dT_{\text{bat}}(t)}{dt} = I_{\text{bat}}(t)^2 R_{\text{int}} - \frac{T_{\text{bat}}(t) - T_{\text{amb}}}{R_{\text{th,bat}}} \tag{6.9}$$

6.4.5 Battery Equivalent Circuit Model (ECM)

Battery terminal voltage $V_{\text{bat}}(t)$ is represented by an ohmic equivalent circuit:

$$V_{\text{bat}}(t) = V_{\text{ocv}}(SOC) - I_{\text{bat}}(t)R_{\text{int}}(SOC, T_{\text{bat}}) \tag{6.10}$$

where open-circuit voltage $V_{\text{ocv}}(SOC)$ scales with state of charge (SOC), and internal resistance R_{int} varies with SOC, temperature, and battery degradation. State of charge evolves via Coulomb counting based on total drawn power and charging currents:

$$\frac{dSOC(t)}{dt} = -\frac{I_{\text{bat}}(t)}{Q_{\text{nom}}} \cdot 100 \tag{6.11}$$

Omission of the polarization branch (V_{pol}):

A complete 1-RC ECM introduces a capacitive polarization branch ($\frac{dV_{\text{pol}}}{dt} = -\frac{V_{\text{pol}}}{R_p C_p} + \frac{I_{\text{bat}}}{C_p}$). This branch is deliberately omitted, and its transfer resistance R_p is folded into R_{int} ($0.035 + 0.015 = 0.050\,\Omega$). *Justification*: Detailed R_p/C_p parameters cannot be calibrated without dedicated electrochemical pulse tests, and the polarization branch accounts for less than 20% of total voltage sag. In a purely ohmic model, the load-step response is directly proportional to $I_{\text{bat}} \cdot R_{\text{int}}$, producing a clean, unambiguous signature for battery state-of-health (SOH) degradation without masking other anomalies.

6.4.6 Physical Fault Injection and Transducer Emulation

Rather than injecting synthetic noise into high-level features, faults perturb the physical equations directly:

- **Mechanical Stall:** Imposes $\omega_m = 0$, collapsing back-EMF ($E_b = 0$) and causing the current to surge to the drive limit, accompanied by rapid thermal rise and battery voltage sag.

- **Wheel Slippage:** Forces $s(t) \to 1$, reducing load torque ($\tau_L \to 0$) while motor speed remains high.
- **Gearbox Degradation:** Injects an additive friction torque τ_{friction}, increasing steady-state current and operating temperature.
- **Stator Fault:** Injects harmonic current ripple and multiplies copper Joule heating.
- **Battery SOH Degradation:** Multiplies R_{int}, producing pronounced voltage drops during acceleration steps.

Finally, physical states are conditioned through transducer models (Gaussian noise, ADC quantization, and thermal/gain drift) to generate realistic diagnostic telemetry for anomaly detection algorithms.

6.4.7 Synthesized Telemetry Channels and Payload Mapping

The physical states computed by the model are serialized into the extended VDA 5050 telemetry payload published by each simulated vehicle. These sensor channels are grouped in a single `information` element of type `motor_telemetry`, whose keys are emitted in a fixed order that forms part of the downstream data contract. The extended telemetry channels incorporated into the simulator are:

- Motor current (I_m) (`motor_current`)
- Motor voltage (V_m) (`motor_voltage`)
- Motor temperature (T_m) (`motor_temp`)
- Motor angular velocity (ω_m) (`motor_speed_rad_s`)
- Motor torque (τ_m) (`motor_torque_nm`)
- Back-electromotive force (E_b) (`back_emf_v`)
- Wheel slip ratio (s) (`wheel_slip`)
- Motor power (P_m) (`motor_power_w`)
- Battery current (I_{bat}) (`battery_current_a`)
- Battery temperature (T_{bat}) (`battery_temp_c`)
- Active fault flag (`fault_active`)
- Active fault types (`fault_types`)

These variables were selected because they are commonly used in predictive maintenance, condition monitoring, and fault diagnosis applications for electrically driven industrial vehicles. The two `fault_*` channels carry the ground-truth identifiers of any injected anomaly, which is useful as a weak label for the detection pipeline. Battery state of charge, state of health and terminal voltage are intentionally not duplicated in this list: they continue to be reported through the standard VDA 5050 `batteryState` object.

6.4.8 AI-Assisted Implementation and Validation Workflow

The extension of the original `vda5050_vehicle_simulator` required the implementation of additional electromechanical, thermal, and battery modelling capabilities based on the physical framework described in the preceding subsections. Since the simulator adaptation itself was not the primary research objective of this work, modern AI-assisted software engineering techniques were employed to accelerate development.

Specifically, an on-premise local AI model provided by the enterprise, NOVA, was primarily utilized during the design and iterative implementation phases, complemented by Google Gemini 3.8 Flash for architectural specification, code validation, and prompt structuring. The models were provided with detailed requirements regarding the desired physical behaviour, software architecture, VDA 5050 compatibility constraints, and telemetry channels to be incorporated into the simulator.

The prompts were formulated as rigorous software specification documents detailing the required electromechanical, thermal, and battery equations, architectural constraints within the asynchronous Tokio runtime, Serde serialization structures, runtime fault scenarios triggered via VDA 5050 Instant Actions, and unit test requirements.

Crucially, the AI-generated code was not incorporated directly into the project without verification. Instead, all algorithmic proposals were systematically reviewed, manually adapted to fit the simulator codebase, validated against theoretical physics curves, and thoroughly tested before final integration. A comprehensive validation of the simulator's physical responses and fault injection behavior is presented in Chapter 7.

6.5 MQTT Broker Implementation and Integration

Architectural counterpart: Chapter 5, Section 5.4.

In the proposed architecture, Message Queuing Telemetry Transport (MQTT) serves as the lightweight edge messaging protocol connecting autonomous mobile robots to the data platform. As specified by the VDA 5050 standard, vehicle state and order messages are published over MQTT topics. To operationalize this layer within the containerized platform, the broker is deployed as a lightweight, containerised ingress service. The companion Telegraf bridge that forwards this edge traffic to the central event streaming backbone is described separately in Section 6.7.

6.5.1 Eclipse Mosquitto Deployment and Broker Configuration

The MQTT broker is implemented using Eclipse Mosquitto, deployed as a containerized service (`mosquitto`) on the dedicated Docker bridge network `mqtt-net`. The broker exposes standard port 1883 for unencrypted edge communication within the container environment.

Listing 6.3 illustrates the broker configuration (`mosquitto.conf`). Persistence is disabled (`persistence false`) because message durability and fault tolerance are delegated to the central event streaming log (Redpanda). All log output is routed to `stdout` to integrate with Docker's logging drivers.

```
persistence false
log_dest stdout

listener 1883 0.0.0.0
```

```
allow_anonymous true
```

Listing 6.3. *Mosquitto configuration (mosquitto.conf)*

To prevent race conditions during stack initialization, a container health check is established in Listing 6.4. The probe uses `mosquitto_pub` to publish a test message to a diagnostic topic every 5 seconds. Upstream telemetry producers (such as the vehicle simulator) and downstream consumers (such as Telegraf) utilize Docker's `condition: service_healthy` dependency directive to ensure the broker is fully responsive before attempting connections.

```
services:
  mosquitto:
    image: eclipse-mosquitto:latest
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto/config:/mosquitto/config
    healthcheck:
      test: ["CMD", "mosquitto_pub", "-h", "localhost", "-p", "1883", "-t", "healthcheck", "-m",
          "1"]
      interval: 5s
      timeout: 5s
      retries: 10
    networks:
      - mqtt-net
```

Listing 6.4. *Mosquitto Docker Compose configuration*

6.6 Event Stream Layer Implementation

Architectural counterpart: Chapter 5, Section 5.5.

The event streaming layer was implemented using Redpanda, a Kafka-compatible event streaming platform that acts as the central event bus connecting the telemetry processing components. Three services are defined in the Docker Compose configuration: the Redpanda broker, Redpanda Console for stream monitoring and visualization, and a one-shot initialization service for creating the required topics.

The Redpanda broker is configured to expose two Kafka-compatible endpoints. The internal endpoint (`redpanda:9092`) enables communication between services within the Docker network, while the external endpoint (`localhost:19092`) allows access from the host machine. The administrative API is exposed on port `9644`, and persistent storage is configured through a Docker volume to preserve broker data across container restarts.

To improve startup reliability, a health check is configured using `rpk cluster health`. The Redpanda Console service waits until the broker is healthy before starting, while the initialization service uses the same dependency condition to ensure that topic creation is attempted only after the broker is operational. The initialization service creates the `vda5050-state`,

`vda5050-state-deduplicated`, `fleet-features`, and `ai-anomalies` topics with four partitions and exits after completing the operation. The `--if-not-exists` option prevents errors when the topic has already been created.

A four-partition configuration was selected to provide parallelism for the expected telemetry workload. Depending on message size, hardware resources, replication settings, and processing overhead, a partition may support throughput in the order of thousands or tens of thousands of messages per second. Therefore, four partitions provide a reasonable initial configuration for the simulated fleet, although the effective capacity must be validated through performance testing. The broker runs in `dev-container` mode with one CPU core assigned through `--smp 1`, reflecting the resource constraints of the local development environment.

```
services:
  redpanda:
    image: docker.redpanda.com/redpandadata/redpanda:v26.1.12
    hostname: redpanda
    command:
      - redpanda
      - start
      - --kafka-addr internal://0.0.0.0:9092,external://0.0.0.0:19092
      - --advertise-kafka-addr internal://redpanda:9092,external://localhost:19092
      - --rpc-addr redpanda:33145
      - --advertise-rpc-addr redpanda:33145
      - --mode dev-container
      - --smp 1
      - --default-log-level=info
    ports:
      - 19092:19092
      - 19644:9644
    volumes:
      - redpanda-data:/var/lib/redpanda/data
    networks:
      - redpanda-net
    healthcheck:
      test: ["CMD-SHELL", "rpk cluster health --exit-when-healthy"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s

  redpanda-console:
    image: docker.redpanda.com/redpandadata/console:v3.8.0
    entrypoint: /bin/sh
```

```
    command: -c 'echo "$$CONSOLE_CONFIG_FILE" > /tmp/config.yml; /app/console'
    environment:
      CONFIG_FILEPATH: /tmp/config.yml
      CONSOLE_CONFIG_FILE: |
        kafka:
          brokers: ["redpanda:9092"]
        redpanda:
          adminApi:
            enabled: true
            urls: ["http://redpanda:9644"]
    ports:
      - 9080:8080
    depends_on:
      redpanda:
        condition: service_healthy
    networks:
      - redpanda-net

  redpanda-init:
    image: docker.redpanda.com/redpandadata/redpanda:v26.1.12
    restart: "no"
    entrypoint: ["/bin/sh", "-c"]
    command:
      - |
        rpk topic create vda5050-state --partitions 4 -X brokers=redpanda:9092 --if-not-exists
        rpk topic create fleet-features --partitions 4 -X brokers=redpanda:9092 --if-not-exists
        rpk topic create ai-anomalies --partitions 4 -X brokers=redpanda:9092 --if-not-exists
    depends_on:
      redpanda:
        condition: service_healthy
    networks:
      - redpanda-net

volumes:
  redpanda-data:
```

Listing 6.5. *Redpanda Docker Compose*

6.7 Streaming Integration and Inter-Service Connectivity

Architectural counterpart: Chapter 5, Section 5.6.

This section consolidates the connector services that bridge the different architectural tiers, mirroring the integration topology described in Chapter 5. Two bridges are implemented: the Telegraf agent that forwards edge MQTT telemetry into Redpanda, and the Kafka Connect cluster that sinks enriched streams from Redpanda into QuestDB.

6.7.1 Telemetry Bridging via Telegraf

Architectural counterpart: Chapter 5, Section 5.6.

A key architectural requirement is the clean decoupling between the edge MQTT protocol and the central Redpanda streaming tier. Because the MQTT boundary only requires protocol adaptation between MQTT and Kafka-compatible streams, Telegraf was selected as a dedicated telemetry ingestion agent.

Telegraf operates as a dual-homed container attached simultaneously to `mqtt-net` and `redpanda-net`. This network topology enforces strict isolation: source systems and the MQTT broker remain in `mqtt-net` with zero direct exposure to the internal Kafka network, while Telegraf transparently forwards incoming payloads across the boundary.

Listing 6.6 shows the Docker Compose configuration for the Telegraf bridging service.

```
services:
  telegraf:
    image: telegraf:1.39-alpine
    restart: unless-stopped
    command: telegraf --config /etc/telegraf/conf/telegraf.conf
    volumes:
      - ./telegraf.conf:/etc/telegraf/conf/telegraf.conf:ro
    environment:
      MQTT_BROKER_URL: ${MQTT_BROKER_URL:-tcp://mosquitto:1883}
      KAFKA_BROKER: ${TELEGRAF_KAFKA_BROKER:-redpanda:9092}
    networks:
      - mqtt-net
      - redpanda-net
```

Listing 6.6. *Telegraf Docker Compose configuration*

The bridging logic is governed by three coordinated stages configured in `telegraf.conf` (Listing 6.7):

1. **MQTT Telemetry Ingestion (`inputs.mqtt_consumer`):** Telegraf subscribes to the VDA 5050 state topic hierarchy using the wildcard pattern `uagv/v2/+/+/state`, matching all vehicles across arbitrary manufacturers. Ingestion uses Quality of Service 1 (QoS 1) with persistent sessions, preventing telemetry loss during momentary network disconnects. Telemetry payloads are ingested as raw string values (`data_format = "value"`) to prevent premature deserialization.

2. **Topic Parsing and Composite Key Generation (`processors.template`):** The hierarchical MQTT topic structure is parsed into semantic tags:

   `_/_/source_manufacturer/source_serial_number/message_type`

A processor template constructs a composite key `source_asset_key` formatted as `<manufacturer>:<serial_number>` (e.g., `rikeb:s11`). This composite key serves as the Kafka message routing key (`routing_tag`), guaranteeing two crucial properties in Redpanda:

- *Collision avoidance*: AGVs from different manufacturers with identical serial numbers are unambiguously partitioned.
- *Chronological partition ordering*: Messages from the same physical asset consistently hash to the same Kafka partition, ensuring strict temporal order for downstream state reconstruction.

3. **Unmutated Kafka Publishing and Header Enrichment (`outputs.kafka`):** Telegraf publishes to the centralized `vda5050-state` topic in Redpanda. Crucially, the VDA 5050 JSON payload is emitted completely unmodified (`template = '{{ .Field "value" }}'`). Schema transformations and data validation are intentionally deferred to downstream Spark Structured Streaming jobs, preserving the raw data fidelity required for the Bronze lakehouse tier. Simultaneously, Telegraf appends enriched Kafka record headers (`source_serial_number`, `source_manufacturer`, `source_asset_key`, `message_type`, and `mqtt_topic`), providing full protocol provenance to consumers without altering message bodies.

```
[agent]
  interval = "1s"
  flush_interval = "1s"
  precision = "1ms"
  omit_hostname = true

[[inputs.mqtt_consumer]]
  servers = ["${MQTT_BROKER_URL}"]
  topics = ["uagv/v2/+/+/state"]
  client_id = "telegraf-vda5050"
  qos = 1
  persistent_session = true
  data_format = "value"
  data_type = "string"

  [[inputs.mqtt_consumer.topic_parsing]]
    topic = "uagv/v2/+/+/state"
    tags = "_/_/source_manufacturer/source_serial_number/message_type"

[[processors.template]]
  order = 0
  tag = "source_asset_key"
  template = '{{ .Tag "source_manufacturer" }}:{{ .Tag "source_serial_number" }}'
```

```
24
25 [[outputs.kafka]]
26   brokers = ["${KAFKA_BROKER}"]
27   topic = "vda5050-state"
28   routing_tag = "source_asset_key"
29   required_acks = -1
30   idempotent_writes = true
31   compression_codec = 4
32   startup_error_behavior = "retry"
33   data_format = "template"
34   template = '{{ .Field "value" }}'
35
36   [outputs.kafka.headers]
37     source_serial_number = '{{ index .Tags "source_serial_number" }}'
38     source_manufacturer = '{{ index .Tags "source_manufacturer" }}'
39     source_asset_key = '{{ index .Tags "source_asset_key" }}'
40     message_type = '{{ index .Tags "message_type" }}'
41     mqtt_topic = '{{ index .Tags "topic" }}'
```

Listing 6.7. *Telegraf bridging configuration (telegraf.conf)*

6.7.2 Kafka Connect Integration and Custom Single Message Transformations

Architectural counterpart: Chapter 5, Section 5.6.

To transfer telemetry seamlessly from the Redpanda broker into QuestDB, a specialized Kafka Connect service (`questdb-kafka-connect`) was implemented. Because QuestDB is an append-only relational time-series database that expects tabular scalar structures, directly ingesting complex, hierarchically nested VDA 5050 JSON payloads (containing nested objects and arrays) poses technical challenges.

(TO DO: SINK CONNECTOR IMPLEMENTATION)

6.8 Stream Processing Layer: Apache Spark Implementation

Architectural counterpart: Chapter 5, Section 5.7. The stream processing tier is implemented using **Apache Spark** (version 3.5.0) via its **Structured Streaming** framework. It is deployed as a set of dedicated containerized services within the Docker Compose orchestration stack.

6.8.1 Docker Compose Service Definition

Listing 6.8 presents the Docker Compose definition of the streaming tier.

Initialization:

The container initialization is governed by health-check and completion dependencies:

- It waits for the `redpanda` broker to report a healthy cluster status.
- It awaits the successful execution of `redpanda-init` to ensure that all required topics (including `vda5050-state` and `vda5050-state-deduplicated`) have been provisioned before job submission.
- It requires `minio` to be started and the `createbuckets` utility container to successfully complete bucket initialization (`lakehouse`).

Once all dependencies are satisfied, streaming execution is launched using `spark-submit` in local client mode (`local[*]`), dynamically resolving the required Kafka integration library (`spark-sql-kafka-0-10`) and Amazon S3A connector (`hadoop-aws`).

```
services:
  spark-vda5050-state:
    image: apache/spark:3.5.0
    container_name: spark-vda5050-state
    depends_on:
      redpanda:
        condition: service_healthy
      redpanda-init:
        condition: service_completed_successfully
      minio:
        condition: service_started
      createbuckets:
        condition: service_completed_successfully
    networks:
      - redpanda-net
      - lakehouse-net
    volumes:
      - ./src:/opt/spark/src
    environment:
      - PYTHONPATH=/opt/spark/src
      - KAFKA_BOOTSTRAP_SERVERS=${KAFKA_BROKER:-redpanda:9092}
      - KAFKA_INPUT_TOPIC=${KAFKA_INPUT_TOPIC:-vda5050-state}
      - S3_ENDPOINT=${S3_ENDPOINT:-http://minio:9000}
      - S3_ACCESS_KEY=${S3_ACCESS_KEY:-admin}
      - S3_SECRET_KEY=${S3_SECRET_KEY:-admin123456}
      - S3_CHECKPOINT_BASE=${S3_CHECKPOINT_BASE:-s3a://lakehouse/checkpoints}
```

```
    - ICEBERG_CATALOG=${ICEBERG_CATALOG:-lakehouse}
    - ICEBERG_WAREHOUSE=${ICEBERG_WAREHOUSE:-s3a://lakehouse/iceberg}
    - ICEBERG_BRONZE_TABLE=${ICEBERG_BRONZE_TABLE:-lakehouse.bronze.vda5050_state}
    - PROCESSING_INTERVAL_BRONZE=${PROCESSING_INTERVAL_BRONZE:-15 seconds}
  command: >
    /opt/spark/bin/spark-submit
    --conf spark.jars.ivy=/tmp/.ivy2
    --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0,org.apache.hadoop:hadoop-aws:3.3.4,
        org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:1.5.2
    --master local[*]
    /opt/spark/src/jobs/vda5050_state_writer.py

spark-fleet-features:
  image: apache/spark:3.5.0
  container_name: spark-fleet-features
  depends_on:
    redpanda:
      condition: service_healthy
    redpanda-init:
      condition: service_completed_successfully
    minio:
      condition: service_started
    createbuckets:
      condition: service_completed_successfully
  networks:
    - redpanda-net
    - lakehouse-net
  volumes:
    - ./src:/opt/spark/src
  environment:
    - PYTHONPATH=/opt/spark/src
    - KAFKA_BOOTSTRAP_SERVERS=${KAFKA_BROKER:-redpanda:9092}
    - KAFKA_INPUT_TOPIC=${KAFKA_INPUT_TOPIC:-vda5050-state}
    - KAFKA_OUTPUT_TOPIC=${KAFKA_OUTPUT_TOPIC:-fleet-features}
    - S3_ENDPOINT=${S3_ENDPOINT:-http://minio:9000}
    - S3_ACCESS_KEY=${S3_ACCESS_KEY:-admin}
    - S3_SECRET_KEY=${S3_SECRET_KEY:-admin123456}
    - S3_CHECKPOINT_BASE=${S3_CHECKPOINT_BASE:-s3a://lakehouse/checkpoints}
    - ICEBERG_CATALOG=${ICEBERG_CATALOG:-lakehouse}
```

```
      - ICEBERG_WAREHOUSE=${ICEBERG_WAREHOUSE:-s3a://lakehouse/iceberg}
      - ICEBERG_SILVER_TABLE=${ICEBERG_SILVER_TABLE:-lakehouse.silver.fleet_features}
      - PROCESSING_INTERVAL_SILVER=${PROCESSING_INTERVAL_SILVER:-3 seconds}
    command: >
      /opt/spark/bin/spark-submit
      --conf spark.jars.ivy=/tmp/.ivy2
      --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0,org.apache.hadoop:hadoop-aws:3.3.4,
          org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:1.5.2
      --master local[*]
      /opt/spark/src/jobs/fleet_features_writer.py

  spark-maintenance:
    image: apache/spark:3.5.0
    restart: unless-stopped
    depends_on:
      spark-vda5050-state:
        condition: service_started
    networks:
      - lakehouse-net
    volumes:
      - ./src:/opt/spark/src
    environment:
      - PYTHONPATH=/opt/spark/src
      - S3_ENDPOINT=${S3_ENDPOINT:-http://minio:9000}
      - S3_ACCESS_KEY=${S3_ACCESS_KEY:-admin}
      - S3_SECRET_KEY=${S3_SECRET_KEY:-admin123456}
      - ICEBERG_CATALOG=${ICEBERG_CATALOG:-lakehouse}
      - ICEBERG_WAREHOUSE=${ICEBERG_WAREHOUSE:-s3a://lakehouse/iceberg}
      - ICEBERG_BRONZE_TABLE=${ICEBERG_BRONZE_TABLE:-lakehouse.bronze.vda5050_state}
      - ICEBERG_SILVER_TABLE=${ICEBERG_SILVER_TABLE:-lakehouse.silver.fleet_features}
      - MAINTENANCE_TARGET=${MAINTENANCE_TARGET:-all}
      - MAINTENANCE_POLL_INTERVAL=${MAINTENANCE_POLL_INTERVAL:-300}
      - MAINTENANCE_SNAPSHOT_THRESHOLD=${MAINTENANCE_SNAPSHOT_THRESHOLD:-600}
      - MAINTENANCE_SIZE_THRESHOLD_KB=${MAINTENANCE_SIZE_THRESHOLD_KB:-500}
    command: python3 /opt/spark/src/jobs/maintenance_watcher.py

networks:
  redpanda-net:
  lakehouse-net:
```

Listing 6.8. *Apache Spark Docker Compose configuration*

Apache Iceberg Catalog Configuration:

Metadata management in the streaming tier relies on a **Hadoop-based Iceberg catalog** rather than on an external catalog service. It is instantiated through the Spark `SparkCatalog` implementation by setting `spark.sql.catalog.lakehouse.type = hadoop` and pointing its `warehouse` to the object-store location `s3a://lakehouse/iceberg`. In this configuration the Iceberg metadata tree (table snapshots, schemas and manifests) is persisted directly as files inside the MinIO bucket through the Hadoop filesystem abstraction, which removes the need for a standalone metastore such as a Thrift or REST catalog and further reduces the number of operational dependencies. Table discovery and atomic metadata commits are therefore resolved against the S3-compatible object store itself, in line with the Lakehouse design presented in Chapter 5 (Section 5.8.2).

6.8.2 Jobs

The streaming layer is implemented through a set of dedicated Spark Structured Streaming jobs, each responsible for a specific stage of the data pipeline. Three streaming data-path jobs compose the Medallion pipeline:

- `vda5050_state_writer`: Persists raw VDA5050 telemetry from Kafka into the Bronze layer.
- `fleet_features_writer`: Extracts, transforms and enriches telemetry into a machine-learning-ready feature representation.
- `ai_anomalies_writer`: Executes real-time anomaly detection and publishes anomaly scores for downstream consumption.

In addition, an auxiliary operational job (`spark-maintenance`) executes asynchronously outside the streaming data flow to perform automated compaction and metadata maintenance on the Iceberg tables. The feature engineering and anomaly detection streaming jobs implement a Dual-Sink Output Pattern, simultaneously persisting processed data into the data lake and publishing the resulting stream back to Redpanda for further processing.

Dual-Sink Output Pattern

The feature engineering and anomaly detection jobs write their outputs to two independent destinations:

- **Analytical Persistence Sink (MinIO):** Processed datasets are stored as Apache Iceberg tables within the data lakehouse, enabling historical analytics and schema evolution.
- **Event Streaming Sink (Redpanda):** Processed records are republished to Kafka topics, allowing downstream consumers to access enriched telemetry and anomaly detection results in real time.

6.8.3 Bronze Ingestion Job

The Bronze ingestion job (`vda5050_state_writer.py`) constitutes the entry point of the Medallion architecture. Its responsibility is to persist the incoming telemetry stream without modification, creating an immutable historical record of the original VDA5050 messages. Unlike subsequent processing stages, no feature extraction, filtering, aggregation or deduplication is applied during Bronze ingestion. The objective of this layer is to preserve the raw telemetry exactly as it was received from the messaging infrastructure, ensuring full data lineage, reproducibility and auditability.

Ingestion and Storage Configuration

The job is implemented using PySpark Structured Streaming and continuously consumes records from the Kafka topic `vda5050-state`. Incoming messages are processed in micro-batches and persisted into an Apache Iceberg table stored in the MinIO object store. Spark is configured to access MinIO through the S3A filesystem connector, using path-style access to the endpoint:`http://minio:9000`.

The target Iceberg table represents the Bronze layer of the data platform and stores the complete telemetry payload together with ingestion metadata required for historical analysis and full data lineage.

Ingestion and Storage Configuration

The job executes micro-batches at a configurable interval, defined by the environment variable `PROCESSING_INTERVAL`.

6.8.4 Silver Feature Engineering Job

Para la implementación de la capa de silver, se crea un job llamado, fleet-features-writer.py, el cual se encarga de flattening y de features extractions. Lee del topic, vda5050-state, hace flattening, y feature extraction, y luego escribe en el topic fleet-features, y en la tabla de iceberg con el mismo nombre. El job reutiliza las funciones de sink.py y de spark. Para el tiempo de procesamiento, se utiliza un microbatch personalizable, el cual se puede configurar a través de la variable de entorno. En el caso de silver, se establece un tiempo de procesamiento por defecto menor al de bronze, debido al requisito de real time para fleet-features, que tiene que ser posteriormente en la parte de inferencia analizado en tiempo real.

(TO DO)

6.8.5 Gold Inference Job

(TO DO)

6.8.6 Lakehouse Maintenance Job

Architectural counterpart: Chapter 5, Section 5.8.3.

Continuous micro-batch ingestion into the Bronze table (every 15 seconds) and Silver table (every 3 seconds) continuously writes small Parquet files and generates rapid metadata commits. Over time, this causes the small files problem and metadata bloat. To counteract this without impacting the latency or memory footprint of the active streaming jobs, lakehouse maintenance is decoupled into a dedicated two-stage operational service:

1. **Smart Observer Daemon (`maintenance_watcher.py`):** A lightweight background daemon executed directly under Python 3 without instantiating an Apache Spark session or JVM. At configurable polling intervals (`MAINTENANCE_POLL_INTERVAL`, default 300 seconds), the daemon queries the MinIO S3 API using standard AWS Signature Version 4 requests in pure Python. It reads the Iceberg `version-hint.text` file to locate the active metadata descriptor (`v{version}.metadata.json`), parses the current snapshot count and computes the JSON metadata size in kilobytes. Maintenance is dynamically triggered only when either of two operational thresholds is breached:

   - Monitored table snapshots $\geq$ `MAINTENANCE_SNAPSHOT_THRESHOLD` (default: 600 snapshots).
   - Metadata descriptor size $\geq$ `MAINTENANCE_SIZE_THRESHOLD_KB` (default: 500 KB).

   This threshold-based observer approach avoids spinning up heavy Spark processes when tables are within optimal parameters, conserving cluster CPU and RAM.

2. **Isolated Compaction Execution (`maintenance.py`):** When a threshold is exceeded, the watcher launches `maintenance.py` as an isolated `spark-submit` subprocess. Subprocess isolation guarantees that the JVM is completely terminated and its memory reclaimed by the operating system upon completion. The Spark job connects to the Hadoop Iceberg catalog and sequentially executes three table maintenance procedures via Spark SQL:

   - `CALL lakehouse.system.rewrite_data_files`: Rewrites and consolidates small micro-batch Parquet files into standardized 64 MiB target files (`target-file-size-bytes = 67108864`), significantly reducing file count and metadata pointers.
   - `CALL lakehouse.system.rewrite_manifests`: Reorganizes and defragments the manifest files to accelerate partition pruning during query planning.
   - `CALL lakehouse.system.expire_snapshots`: Purges obsolete snapshot entries and deletes unreferenced historical Parquet files superseded by compaction (`retain_last = 2`, `older_than = CURRENT_TIMESTAMP`), bounding object storage volume while keeping the active snapshot dataset completely intact.

The job accepts a configurable target via `MAINTENANCE_TARGET` (`all`, `bronze`, or `silver`), defaulting to maintaining both layers of the Medallion lakehouse.

6.9 Data Lakehouse Layer Implementation

Architectural counterpart: Chapter 5, Section 5.8.

(TO BE COMPLETED: Overview of the Data Lakehouse implementation, storage topology, and integration between object storage and modern table formats).

6.9.1 S3-Compatible Object Storage: MinIO

The object storage tier was implemented using **MinIO**, an open-source, high-performance distributed object storage server fully compatible with the Amazon S3 API. In this architecture, MinIO serves as the primary persistence layer for storing raw Bronze Parquet files and managing streaming checkpoints for Apache Spark.

Two services are defined within the Docker Compose deployment:

- **MinIO Server (`minio`):** Runs the core object storage engine, persisting data in a dedicated Docker volume (`minio-data`). The S3 API operates on container port 9000, which is mapped to host port 19000 to prevent host port collisions with QuestDB. The interactive web console is exposed on host port 9001. A container health check probes `/minio/health/live` to guarantee service availability before downstream consumers initiate requests.
- **Bucket Provisioning (`createbuckets`):** A lightweight initialization container using the MinIO Client (`minio/mc`) CLI. It waits for the MinIO server to report a healthy status, configures a local connection alias, and automatically provisions the target bucket (`lakehouse`) using `mc mb --ignore-existing`. Once bucket creation succeeds, the container terminates cleanly.

Both services are attached to the `lakehouse-net` network, ensuring secure, isolated network communication with the Spark stream processing tier.

```
services:
  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: admin
      MINIO_ROOT_PASSWORD: admin123456
    volumes:
      - minio-data:/data
    ports:
      - "19000:9000"
      - "9001:9001"
    networks:
      - lakehouse-net
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
      interval: 10s
      timeout: 5s
      retries: 5

  createbuckets:
    image: minio/mc
    depends_on:
      minio:
        condition: service_healthy
    entrypoint: >
      /bin/sh -c "
      sleep 5;
```

```
      mc alias set local http://minio:9000 admin admin123456;
      mc mb --ignore-existing local/lakehouse;
      exit 0;
      "
    networks:
      - lakehouse-net

volumes:
  minio-data:
```

Listing 6.9. *MinIO Docker Compose configuration*

6.9.2 Table Format Implementation: Apache Iceberg

(TO BE COMPLETED: Apache Iceberg catalog configuration, table schema definition, and partitioning strategy across Bronze, Silver, and Gold tiers).

6.10 QuestDB Implementation and Integration

Architectural counterpart: Chapter 5, Section 5.8.4.

As established in the architectural design (Chapter 5), the platform adopts a bifurcated persistence strategy that strictly separates the analytical lakehouse storage (MinIO and Apache Iceberg) from the operational time-series database. While the lakehouse is optimized for heavy, multi-tier batch analytics and offline model training, real-time observability requires a database capable of sub-second streaming ingestion and high-frequency SQL aggregations across sliding time windows. To fulfill this operational role, **QuestDB** (version 9.4.3) was selected and integrated into the containerized platform.

The QuestDB subsystem interfaces directly with the central event streaming layer through a dedicated, containerized **Apache Kafka Connect** cluster. This design establishes an automated ingestion pipeline that streams live VDA 5050 robot telemetry from Redpanda into QuestDB without requiring intermediate message-polling services or bespoke application logic.

6.10.1 Database Deployment and Schema Architecture

The database engine is deployed as an isolated service (`questdb`) within a dedicated bridge network (`questdb-net`). Persistent telemetry is stored on a dedicated Docker volume (`questdb-data`), preserving all historical state messages across container restarts.

QuestDB provides native multi-protocol support. In this deployment, three distinct interfaces are configured:

- **HTTP REST API and Web Console (port 9000):** Used for system administration, automated schema provisioning, and diagnostic telemetry inspection.
- **PostgreSQL Wire Protocol (port 8812):** QuestDB natively emulates the PostgreSQL wire protocol. To adhere to the principle of least privilege, a dedicated read-only user (`QDB_PG_READONLY_USER: grafana`) is configured. This

endpoint allows Grafana dashboards to query operational tables directly using standard SQL without exposing write permissions.

- **InfluxDB Line Protocol (port 9009):** Maintained for low-overhead, high-throughput streaming metrics ingestion if required.

A container health check continuously probes the HTTP status endpoint (`http://localhost:9003/status`) to ensure the engine is fully initialized before downstream connectors attempt data transmission.

To automate database table provisioning upon initial startup, a one-shot container (`questdb-init`) is defined using `curlimages/curl`. It executes after QuestDB reports a healthy state, submitting SQL DDL commands to QuestDB's `/exec` REST endpoint.

(TO DO: TABLE IMPLEMENTATIONS)

6.10.2 Automated Lifecycle and Dependency Management

To guarantee reliable system initialization, service orchestration is strictly ordered through Docker health checks and one-shot provisioning containers:

1. The `questdb` service initializes its engine and validates readiness on port 9003.
2. The `questdb-init` service triggers the SQL DDL creation of the `vda5050_state` table.
3. The `questdb-kafka-connect` service initializes only after the Redpanda broker is healthy, all required Kafka topics are created (`redpanda-init`), and QuestDB tables exist (`questdb-init`).
4. Finally, a dedicated initialization container (`questdb-kafka-connect-init`) executes an automated shell script that submits the connector configuration via the Kafka Connect REST API (`PUT /connectors/<name>/config`). The script actively polls the connector status endpoint until the task achieves the `RUNNING` state, ensuring that the pipeline is completely operational before simulated AGVs begin streaming messages.

This architecture guarantees end-to-end automated orchestration, preventing race conditions during platform startup while maintaining strict network isolation between operational storage, event streaming, and analytical tiers.

(TO DO: DOCKER COMPOSE)

6.11 Machine Learning Pipeline Implementation

Architectural counterpart: Chapter 5, Section 5.9.

(TO DO)

6.12 Grafana Implementation

Architectural counterpart: Chapter 5, Section 5.10.

(TO DO)

6.13 Security and Network Isolation Enforcement

Architectural counterpart: Chapter 5, Section 5.11.

(TO DO)

7

Testing and Evaluation

7.1 Introduction

This chapter evaluates the functional fidelity, operational resilience, and throughput performance of the proposed predictive maintenance and anomaly detection platform. The evaluation assesses both the physical accuracy of the generated telemetry and the stability of the distributed streaming lakehouse under sustained, real-time industrial workloads. *(To complete: additional evaluation sections and machine learning validation procedures will be incorporated as the platform matures.)*

Experimental Test Environment and Reproducibility.

To guarantee strict environment parity, reproducibility, and process isolation, the entire data platform is deployed as a containerized stack orchestrated via Docker Compose. Every functional component—including the native Rust-based vehicle simulator, the Eclipse Mosquitto MQTT broker, the Telegraf ingestion bridge, the Redpanda distributed streaming backbone, the Apache Spark Structured Streaming engines, the Apache Iceberg lakehouse on MinIO S3, the QuestDB time-series database, and the Grafana visualization service—executes within independent container boundaries connected through dedicated software-defined bridge networks.

All experimental tests and soak evaluations reported in this chapter were carried out on a dedicated physical workstation. Table 7.1 summarizes the hardware specifications of the host machine and the container execution runtime.

Table 7.1. *Experimental Host Hardware and Container Execution Environment*

System Parameter	Specification
Host Processor (CPU)	Intel(R) Core(TM) Ultra 7 265H (16 physical cores, x86_64)
CPU Cache Hierarchy	24 MiB L3 cache, 48 MiB L2 cache, 1.75 MiB L1 cache
Physical Host Memory	64 GB physical RAM
Dedicated Container RAM	31 GiB allocated memory with 8.0 GiB swap (WSL2 managed)
Local Storage	1 TB NVMe SSD (`ext4` virtual disk)
Host Operating System	Microsoft Windows 11 Enterprise / Professional
Linux Runtime Subsystem	Ubuntu 24.04.4 LTS (Noble Numbat) on WSL2 (Kernel 6.6.87)
Container Engine	Docker Engine 29.5.3-rd
Orchestration Tool	Docker Compose v5.1.4

The virtualization architecture utilizes the Windows Subsystem for Linux (WSL2) backed by a lightweight Hyper-V hypervisor. Unlike traditional full-system virtual machines, WSL2 provides direct native hardware access to all 16 physical CPU cores without core pinning overhead, while dynamically allocating a bounded 31 GiB RAM slice dedicated to Docker containers. This ensures that the compute and memory benchmarks faithfully represent real-world bare-metal industrial edge deployment conditions.

7.2 Testing: testing adjusted simulator

The physical powertrain model formulated in Section 6.4 and incorporated into the simulator as described in Section 6.4.7 replaces heuristic signal generation with an explicit electromechanical model: an armature circuit, a torque balance referred to the motor shaft, a lumped thermal node for the winding, a battery with an equivalent-circuit terminal voltage and a set of transducers that inject noise, quantisation and drift into every published channel. Introducing that model raises a question that unit tests alone cannot answer: whether the telemetry that finally reaches the historian still *looks* like the physical system it claims to represent.

Nature of this test. The checks collected in this section are *functional, informational tests* executed by **manual inspection** of the telemetry rendered in the Grafana dashboards. They are not automated tests, they are not executed by a CI/CD pipeline, they produce no pass/fail artefact and they are not regression protected: every run is observed, interpreted and archived by hand. Their purpose is evidential rather than verificational, to show a reader of the thesis that the simulated signals reproduce the qualitative and quantitative behaviour expected from the electromechanical model, so that the anomalies synthesised later on can be argued to be physically plausible and not artefacts of arbitrary signal shaping.

7.2.1 Test Environment and Procedure

All observations were taken from the deployed stack: the simulated fleet publishes one VDA 5050 `state` message per robot and second, which travels through the MQTT broker through Kafka Connect into the QuestDB table `vda5050_state`, where the Grafana dashboards, representations of each source form read them. The state publication rate of one hertz is the effective sampling rate of every observation reported below and sets the limit of what can be seen. A gaussian noise is introduce to simulate the real world behaviour.

7.2.2 Motor voltage and governing variables (Vm, Im, wm, Ra, La, Ke)

To verify that the simulator adheres to the electromechanical motor model, the quasi-steady armature equation $V_m(t) \approx R_a(T_m) \cdot I_m(t) + K_e \cdot \omega_m(t)$ (neglecting the fast inductive transient $L_a \frac{dI_m}{dt} \approx 0$) was evaluated using telemetry sampled during cruise motion. With an armature resistance $R_a = 0.0501\,\Omega$, current $I_m = 2.9688\,\text{A}$, rotor speed $\omega_m = 100.0413\,\text{rad/s}$, and $K_e = 0.19\,\text{V} \cdot \text{s/rad}$, the calculated back-EMF is $E_b = K_e \cdot \omega_m = 19.0078\,\text{V}$ (closely matching the measured $E_{b,\text{meas}} = 18.9922\,\text{V}$) and the ohmic drop is $R_a \cdot I_m = 0.1487\,\text{V}$. The theoretical terminal voltage yields $V_{m,\text{calc}} = 0.1487\,\text{V} + 18.9922\,\text{V} = 19.1409\,\text{V}$, which deviates by only $0.0550\,\text{V}$ (a relative error of 0.28%) from the measured $V_{m,\text{meas}} = 19.0859\,\text{V}$. This minor discrepancy is completely bounded by the simulated 12-bit ADC quantization step ($\approx 7.8\,\text{mV}$) and the Gaussian sensor noise ($\sigma_v = 0.03\,\text{V}$), confirming that the simulator accurately follows the physical motor principle.



Figure 7.1. *Motor voltage form, with all the terms*

7.2.3 Motor Torque and governing variables (τ_m, K_t, I_m)

To verify that the simulator accurately governs electromagnetic torque generation, the linear torque-current relationship $\tau_m(t) = K_t \cdot I_m(t)$ of Equation (6.3) was evaluated against the Grafana telemetry panel shown in Figure 7.2. At the sampled operating point (t = 15:56:38, during an idle/standby interval before motion), the torque constant is $K_t = 0.1900\,\text{N} \cdot \text{m/A}$ (numerically equivalent to the back-EMF constant $K_e = 0.1900\,\text{V} \cdot \text{s/rad}$) and the armature current is $I_m = 0.0391\,\text{A}$. The calculated electromagnetic torque yields $\tau_{m,\text{calc}} = K_t \cdot I_m = 0.1900\,\text{N} \cdot \text{m/A} \times 0.0391\,\text{A} = 0.007429\,\text{N} \cdot \text{m}$, matching the dashboard's calculated channel (`calculated_torque_nm` = $0.0074\,\text{N} \cdot \text{m}$) to four decimal places. The measured motor torque reported by the drive telemetry is $\tau_{m,\text{meas}} = 0.0077\,\text{N} \cdot \text{m}$ (`motor_torque_nm`), resulting in an absolute discrepancy of only $0.0003\,\text{N} \cdot \text{m}$ ($0.3\,\text{mN} \cdot \text{m}$). This minute deviation is completely bounded by the 12-bit ADC quantization resolution and the Gaussian transducer noise ($\sigma_I \approx 0.08\,\text{A}$, translating to $\sigma_\tau \approx 0.015\,\text{N} \cdot \text{m}$). Across the entire mission profile, dynamic tracking remains consistent: during acceleration ramps, current surges to $I_m \approx 8.63\,\text{A}$ producing a torque peak of $\approx 1.63\,\text{N} \cdot \text{m}$, whereas during steady-state cruise at $I_m \approx 3.0$–$3.2\,\text{A}$, torque settles stably at ≈ 0.57–$0.61\,\text{N} \cdot \text{m}$. The measured and calculated torque traces overlap seamlessly throughout all motion phases, confirming that the simulated machine faithfully reproduces electromagnetic torque conversion without synthetic distortions.



***Figure 7.2**. Motor torque form, with all the terms*

7.2.4 Mechanical torque balance and load dynamics (τ_m, τ_L, ω_m, B_m, J_m)

The dynamic torque equilibrium referred to the motor rotor shaft is governed by Equation (6.4), $J_m \frac{d\omega_m(t)}{dt} = \tau_m(t) - B_m \cdot \omega_m(t) - \tau_{L,\text{ref}}(t)$, or expressed in terms of the total mechanical demand: $\tau_{\text{demand}}(t) = J_m \frac{d\omega_m(t)}{dt} + B_m \cdot \omega_m(t) + \tau_{L,\text{ref}}(t)$, where $\tau_{\text{viscous}}(t) = B_m \cdot \omega_m(t)$ is the internal viscous friction torque and $\tau_{L,\text{ref}}(t) = \frac{\tau_L(t)}{\eta_g i_g}$ represents the ground load torque reflected through the reduction gearbox ($i_g = 10$, $\eta_g = 0.85$). This formulation was verified using the Grafana telemetry panel in Figure 7.3 sampled during steady cruise motion ($t = 15{:}57{:}16$). At this operating point, the rotor speed is stabilized at $\omega_m = 99.9031$ rad/s (corresponding to the vehicle travelling at cruise velocity $v \approx 1.0$ m/s), rendering the angular acceleration null ($\frac{d\omega_m}{dt} \approx 0$) and nullifying the inertial demand ($J_m \dot{\omega}_m \approx 0$). The reported viscous friction torque is $\tau_{\text{viscous}} = 0.1499\,\text{N} \cdot \text{m}$, which confirms a viscous damping coefficient $B_m = \frac{\tau_{\text{viscous}}}{\omega_m} = \frac{0.1499}{99.9031} \approx 0.0015\,\text{N} \cdot \text{m} \cdot \text{s/rad}$. The reflected load torque is $\tau_{L,\text{ref}} = 0.4155\,\text{N} \cdot \text{m}$, corresponding to the rolling resistance of the $180\,\text{kg}$ vehicle on level ground ($F_{\text{roll}} = C_{rr} m_{\text{total}} g = 0.02 \times 180\,\text{kg} \times 9.81\,\text{m/s}^2 \approx 35.32\,\text{N}$, giving $\tau_L = F_{\text{roll}} \cdot r_w \approx 3.532\,\text{N} \cdot \text{m}$ at wheel radius $r_w = 0.10\,\text{m}$, and $\tau_{L,\text{ref}} = \frac{3.532}{0.85 \times 10} \approx 0.4155\,\text{N} \cdot \text{m}$). The theoretical total mechanical demand thus evaluates to $\tau_{\text{demand}} = \tau_{L,\text{ref}} + \tau_{\text{viscous}} = 0.4155\,\text{N} \cdot \text{m} + 0.1499\,\text{N} \cdot \text{m} = 0.5654\,\text{N} \cdot \text{m}$, perfectly coinciding with the reported channel `total_mechanical_demand_nm` ($0.5654\,\text{N} \cdot \text{m}$). The electromagnetic motor torque delivered by the drive at this moment is $\tau_{m,\text{meas}} = 0.6130\,\text{N} \cdot \text{m}$, presenting a minor difference of $0.0476\,\text{N} \cdot \text{m}$ attributable to simulated current transducer noise ($\sigma_\tau \approx 0.015\,\text{N} \cdot \text{m}$) and speed controller regulation ripple. During acceleration ramps, the inertial term $J_m \dot{\omega}_m$ momentarily elevates the torque demand to $\approx 1.63\,\text{N} \cdot \text{m}$, validating that the simulator accurately integrates both static resistive loads and dynamic inertial terms.

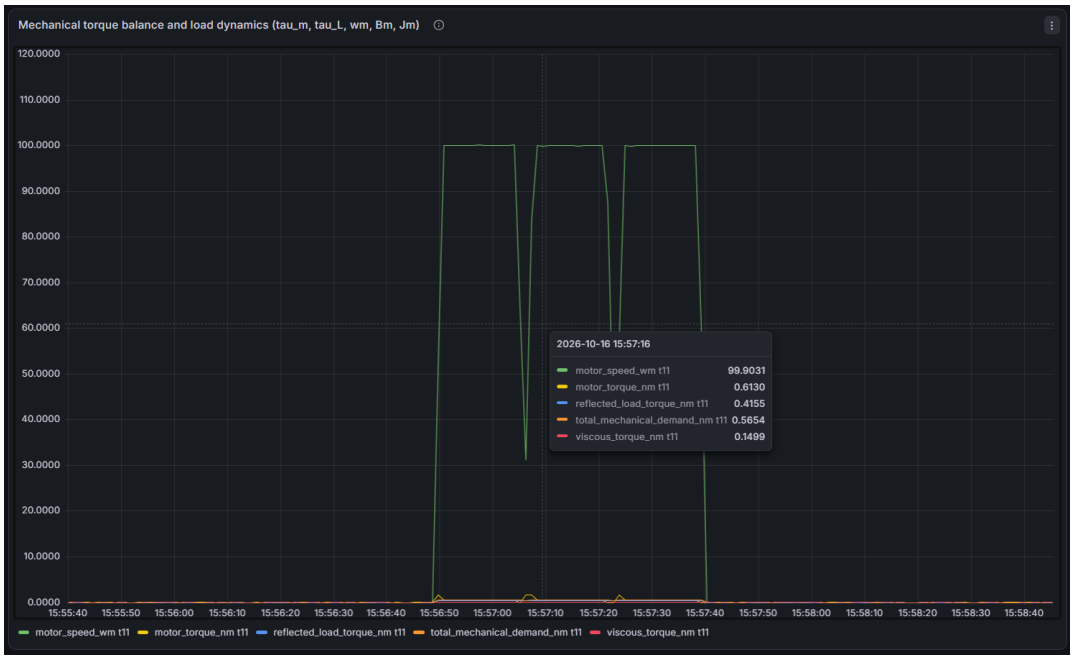


Figure 7.3. *Mechanical torque balance and load dynamics, with all the terms*

7.2.5 Kinematic closure and wheel slip dynamics (v, ω_m, s, r_w, i_g)

The kinematic coupling between the electric motor shaft, transmission, driven wheels, and vehicle ground translation is defined by the kinematic closure relation $v(t) = \frac{\omega_m(t)}{i_g} \cdot r_w \cdot (1 - s(t))$, where $v_w(t) = \frac{\omega_m(t)}{i_g} \cdot r_w$ represents the theoretical peripheral linear velocity of the wheel, $i_g = 10$ is the transmission reduction ratio, $r_w = 0.10\,\text{m}$ is the nominal wheel radius, and $s(t)$ is the longitudinal wheel slip ratio. This formulation was validated using the Grafana telemetry panel shown in Figure 7.4 sampled during steady cruise motion ($t = 15{:}56{:}57$). At this operating point, the reported wheel linear speed is $v_w = 1.0011\,\text{m/s}$ (corresponding to an angular motor velocity $\omega_m = \frac{v_w i_g}{r_w} = 100.11\,\text{rad/s}$), and the estimated longitudinal wheel slip is $s = -0.0073$ (a slight negative micro-slip of -0.73% induced by transducer Gaussian noise and nominal tire contact compliance). Evaluating the kinematic closure equation yields a theoretical ground speed of $v_{\text{calc}} = v_w \cdot (1 - s) = 1.0011\,\text{m/s} \cdot (1 - (-0.0073)) = 1.0084\,\text{m/s}$, matching the dashboard's calculated channel (`calculated_ground_speed_v` = $1.0084\,\text{m/s}$) to four decimal places. When compared to the measured ground speed $v_{\text{meas}} = 1.0000\,\text{m/s}$ (`ground_speed_-v`), the absolute discrepancy is only $0.0084\,\text{m/s}$ (a relative error of 0.84%). Across the mission profile, the wheel speed tracks the ground translation with minimal slip ($s \approx 0$) during unperturbed cruise, while the brief velocity reductions at navigation waypoints are tracked synchronously by both channels. This confirms that the transmission kinematics and slip interaction are physically consistent and free of numerical drift.

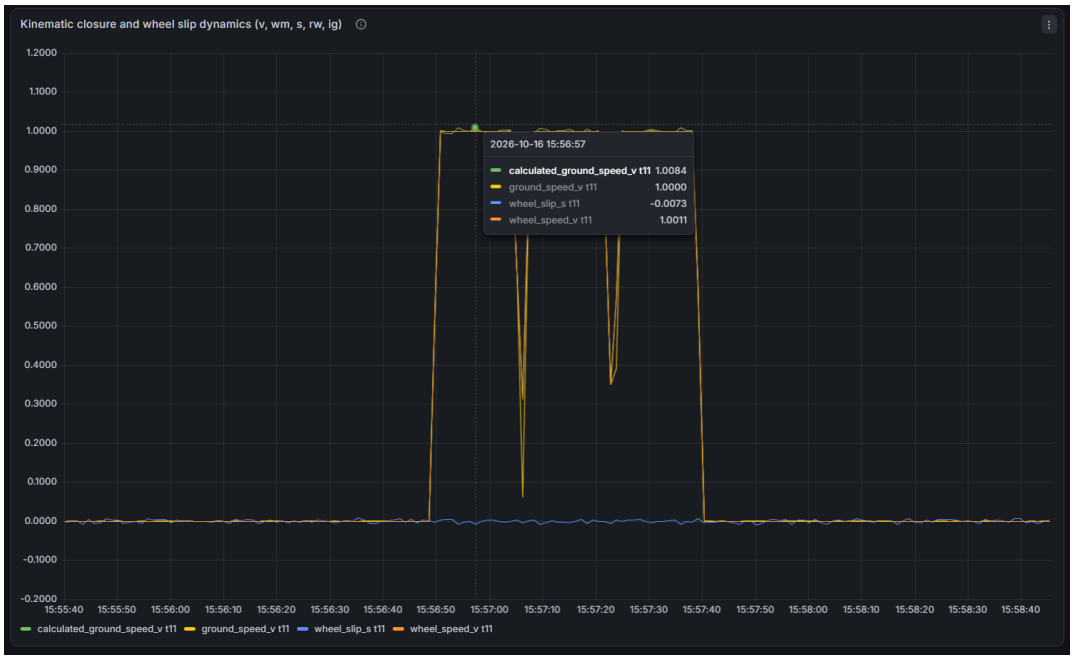


Figure 7.4. *Linear velocity form, with all the terms*

7.2.6 Motor thermal dynamics and power loss balance (T_m, T_{amb}, P_{joule}, P_{iron}, P_{cool})

The thermal evolution of the motor winding is governed by the lumped first-order power balance $C_{th}\frac{dT_m(t)}{dt} = P_{\text{joule}}(t) + P_{\text{iron}}(t) - P_{\text{cool}}(t)$, where $P_{\text{joule}}(t) = I_m(t)^2 R_a(T_m)$ represents the resistive copper losses, $P_{\text{iron}}(t) = C_{fe}\omega_m(t)^{1.6}B_{\text{max}}^2$ represents core iron losses, and $P_{\text{cool}}(t) = \frac{T_m(t)-T_{\text{amb}}}{R_{th}}$ is the convective heat transfer to the ambient environment ($T_{\text{amb}} = 22.0000\,^\circ\text{C}$). This thermal balance was evaluated using the telemetry panel shown in Figure 7.5 at $t =$ 15:57:07 during an acceleration transition between navigation nodes. At this instant, the telemetry records copper losses of $P_{\text{joule}} = 3.6831\,\text{W}$ and core iron losses of $P_{\text{iron}} = 3.7758\,\text{W}$, yielding a total heat dissipation rate of $P_{\text{total}} = 3.6831\,\text{W} + 3.7758\,\text{W} = 7.4589\,\text{W}$. With the winding temperature at $T_m = 23.1900\,^\circ\text{C}$, the temperature rise above ambient is $T_m - T_{\text{amb}} = 1.1900\,\text{K}$, producing a cooling power of $P_{\text{cool}} = 0.4760\,\text{W}$, which exactly identifies the thermal resistance $R_{th} = \frac{T_m - T_{\text{amb}}}{P_{\text{cool}}} = \frac{1.1900\,\text{K}}{0.4760\,\text{W}} = 2.5000\,\text{K/W}$. The instantaneous asymptotic equilibrium temperature is given by $T_\infty = T_{\text{amb}} + P_{\text{total}} \cdot R_{th} = 22.0000\,^\circ\text{C} + (7.4589\,\text{W} \times 2.5000\,\text{K/W}) = 40.64725\,^\circ\text{C} \approx 40.6473\,^\circ\text{C}$, which matches the dashboard channel `equilibrium_temp_c` ($40.6473\,^\circ\text{C}$) to four decimal places. The net thermal power $P_{\text{net}} = P_{\text{total}} - P_{\text{cool}} = 6.9829\,\text{W}$ is integrated across the thermal capacitance $C_{th} = 80\,\text{J/K}$, producing a smooth, continuous exponential temperature rise ($\frac{dT_m}{dt} \approx 0.087\,\text{K/s}$) without artificial discontinuities, verifying the physical validity of the thermal node.



Figure 7.5. *Motor thermal dynamic and power loss balance, with all the terms*

7.2.7 Battery terminal voltage and internal-resistance drop (V_{bat}, V_{ocv}, R_{int}, I_{bat})

The terminal voltage of the traction battery is governed by the simplified equivalent-circuit model (ECM) of Equation (6.10): $V_{\text{bat}}(t) = V_{\text{ocv}}(\text{SOC}) - \Delta V_{\text{ohmic}}(t)$, where $\Delta V_{\text{ohmic}}(t) = I_{\text{bat}}(t) \cdot R_{\text{int}}(\text{SOC}, T_{\text{bat}})$ represents the instantaneous ohmic drop across the pack internal resistance. This relation was examined using the Grafana telemetry panel shown in Figure 7.6 sampled during cruise motion (t = 15:57:11). At this point, the pack delivers a current of $I_{\text{bat}} = 3.0273\,\text{A}$ and exhibits an internal resistance of $R_{\text{int}} = 0.0535\,\Omega$. The theoretical ohmic sag evaluates to $\Delta V_{\text{ohmic}} = I_{\text{bat}} \cdot R_{\text{int}} = 3.0273\,\text{A} \times 0.0535\,\Omega = 0.16196\,\text{V} \approx 0.1619\,\text{V}$, exactly matching the dashboard's reported channel (`ohmic_sag_v` = 0.1619 V). Subtracting this drop from the open-circuit voltage $V_{\text{ocv}} = 23.2248\,\text{V}$ yields a theoretical terminal voltage of $V_{\text{bat,calc}} = 23.2248\,\text{V} - 0.16196\,\text{V} = 23.06284\,\text{V} \approx 23.0629\,\text{V}$, coinciding with `v_bat_calc` (23.0629 V) to four decimal places. The measured terminal voltage reported by telemetry is $V_{\text{bat,meas}} = 23.0938\,\text{V}$ (`v_bat_meas`), differing by only $0.0309\,\text{V}$ (a relative error of 0.13%). This minor discrepancy is completely bounded by the simulated 12-bit ADC quantization step ($\approx 7.8\,\text{mV}$) and the battery voltage sensor Gaussian noise ($\sigma_v = 0.03\,\text{V}$). Dynamically, the panel demonstrates the expected electrochemical response: while idling ($I_{\text{bat}} \approx 0.5\,\text{A}$), the ohmic drop remains minimal and $V_{\text{bat}} \approx V_{\text{ocv}}$; under cruise load ($I_{\text{bat}} \approx 3.0\,\text{A}$), terminal voltage experiences a sustained sag of $\approx 0.16\,\text{V}$; and during acceleration surges ($I_{\text{bat}} \approx 6.8\,\text{A}$), the ohmic dip deepens to $\approx 0.36\,\text{V}$ before recovering, fully validating the ECM implementation.



Figure 7.6. *Battery tension form, with all terms*

7.2.8 Interpretation and Limitations

The values quoted above are consistency checks against the model, not metrological measurements: every one of them is conditioned by the calibration constants published in the simulator configuration, and they hold for the operating point described, a one-hundred-and-eighty-kilogram vehicle travelling unloaded on level ground at 1.0 m/s. Changing the payload, the grade or the speed moves the numerical values while leaving the relations intact, which is precisely the property the checks exercise.

The one hertz publication interval also bounds the scope of the section. Electromagnetic transients of tens of milliseconds, the ripple harmonics injected by the stator short-circuit fault, and the slow drift of the thermistor and the current transducer, specified in fractions of a degree or a percent per hour, are outside what can be observed here; they are asserted deterministically in the automated test suite of the simulator instead. This section is consequently an informational, human-reviewed characterisation of the simulated telemetry, whose role in this work is to justify that the data on which the anomaly detection models are trained and evaluated behaves like a powertrain.

7.3 Evaluation: Data pipeline evaluation

This section assesses the operational resilience, throughput stability, and resource consumption of the end-to-end industrial telemetry streaming pipeline under continuous real-time load. In distributed IoT streaming architectures, standard unit and integration tests are insufficient to guarantee production readiness; long-running *soak tests* are indispensable to expose insidious failure modes such as progressive memory leaks, queue backpressure, partition fragmentation, and metadata accumulation.

7.3.1 Evaluation Methodology and Workload Design

The evaluation campaign was designed to monitor the complete containerized platform over an extended multi-day lifecycle. Across the entire experimental period, the cluster operated without interruption for **62.35 hours (2.6 days)**—spanning from the first message published on 2026-11-09 at 17:29:29 UTC to 2026-11-12 at 07:50:17 UTC.

Synthetic Industrial Workload.

The streaming workload was continuously produced by a simulated fleet of 5 mobile industrial robots (AGVs/AMRs) operating with the electromechanical vehicle simulator described in Section 6.4. Each simulated robot published state messages over MQTT at a constant frequency of 1 Hz (totaling a continuous platform ingestion rate of 5 messages/second). Messages traveled through the Eclipse Mosquitto broker, were bridged by Telegraf into the Redpanda `vda5050-state` topic, transformed in real time by Apache Spark Structured Streaming, persisted into the Apache Iceberg lakehouse on MinIO S3, and ingested into QuestDB for real-time dashboard visualization.

Unattended Overnight Soak Monitoring Protocol.

To eliminate artificial interference caused by interactive desktop usage and operator queries, the empirical auditing sessions were conducted during unattended overnight windows when the workstation was left dedicated entirely to continuous ingestion:

1. **Phase 1 (Day 1 Baseline Soak, 14.30 Hours):** An initial overnight audit (from 2026-11-09 18:31:02 to 2026-11-10 08:48:49 UTC, $N = 1,641$ snapshots) executed under default streaming settings. This phase established the baseline performance and successfully uncovered critical operational vulnerabilities.
2. **Phase 2 (Days 2–3 Extended Validation Soak, 38.86 Hours):** Following daytime root-cause analysis and hardening deployment, an extended multi-day overnight soak test was launched (from 2026-11-10 17:59:59 to 2026-11-12 08:49:44 UTC, $N = 4,450$ snapshots). This phase monitored the hardened platform continuously across two consecutive nights and the intervening day to confirm asymptotic steady-state stability.

Throughout both sessions, an automated background daemon captured discrete `docker stats` snapshots every 30 seconds across all 11 containerized services, recording CPU utilization, resident set size (RSS) memory, network I/O, block I/O, and thread counts.

7.3.2 Phase 1: Baseline Soak Test and Bottleneck Discovery (Day 1)

Objective.

The primary objective of the initial baseline soak test was to verify whether the streaming pipeline could sustain continuous processing without worker crashes, pipeline freezes, or message lag accumulation under default Apache Iceberg configurations.

Baseline Empirical Findings.

During the 14.30-hour initial run, the platform achieved 100% service availability without crashes, restarts, or unhandled exceptions. Ingested data integrity was verified at the storage sinks: exactly 233,412 rows were persisted into QuestDB (`fleet_features`) with uninterrupted timestamps. Furthermore, Apache Spark maintained strict real-time parity with Redpanda without backpressure:

$$\text{avgOffsetsBehindLatest} = 0.0, \quad \text{maxOffsetsBehindLatest} = 0 \tag{7.1}$$

These streaming metrics quantify consumer lag by tracking the offset difference between the latest message published to the broker and the offset committed by Spark. A zero value across the entire soak test proves that micro-batches were processed instantaneously upon arrival without latency drift.

Table 7.2 details the container resource utilization recorded across the 14.30-hour baseline execution.

Table 7.2. *Container Resource Utilization and Memory Drift (Phase 1: Day 1 Baseline Soak Test)*

Service / Container	CPU_{avg}	CPU_{max}	MEM_{avg}	MEM_{max}	MEM_{Start}	MEM_{End}	Δ MEM	Observed Trend
`spark-fleet-features`	24.3%	131.9%	2.1 GiB	3.1 GiB	2.9 GiB	1.7 GiB	-1.2 GiB	Stabilized (GC)
`spark-vda5050-state`	2.5%	48.5%	1.6 GiB	2.1 GiB	2.1 GiB	1.3 GiB	-767 MiB	Stabilized (GC)
`questdb`	12.8%	27.2%	569 MiB	741 MiB	711 MiB	425 MiB	-287 MiB	Decreased (Stable)
`redpanda`	1.3%	9.1%	1.8 GiB	2.1 GiB	1.9 GiB	1.4 GiB	-515 MiB	Stabilized
`minio`	5.8%	33.2%	1.2 GiB	2.1 GiB	400 MiB	2.0 GiB	**+1.6 GiB**	**+406% (Metadata Cache)**
`questdb-kafka-connect`	0.9%	15.2%	581 MiB	939 MiB	616 MiB	476 MiB	-141 MiB	Stabilized
`grafana`	0.6%	6.3%	150 MiB	173 MiB	150 MiB	148 MiB	-1 MiB	Flat
`simulator` (Rust AGV)	0.5%	1.1%	6.1 MiB	6.7 MiB	4.2 MiB	6.3 MiB	+2 MiB	Negligible (Flat)
`telegraf`	0.2%	1.5%	60 MiB	107 MiB	72 MiB	44 MiB	-28 MiB	Stabilized
`mosquitto`	0.5%	5.9%	6.9 MiB	12 MiB	7.9 MiB	6.4 MiB	-1 MiB	Flat
`sim-control-panel`	0.1%	0.2%	27 MiB	28 MiB	25 MiB	26 MiB	+1 MiB	Flat

Critical Bottleneck Discovery: Unbounded Metadata Accumulation.

While computational streaming components behaved stably, post-test inspection revealed severe operational anomalies centered on the storage tier:

1. **Monotonic MinIO RAM Growth (+406%):** As illustrated in Figure 7.7, MinIO resident memory diverged sharply from all other services, climbing continuously from 400 MiB up to 2.07 GiB (+1.6 GiB).

2. **Object Storage Exhaustion (128 GiB):** The MinIO bucket reached 128 GiB of disk usage. Crucially, actual persisted Parquet telemetry data totaled only 321 MB; the remaining **127.5 GiB (99.7% of the entire bucket)** consisted of over 61,000 Apache Iceberg metadata JSON files.

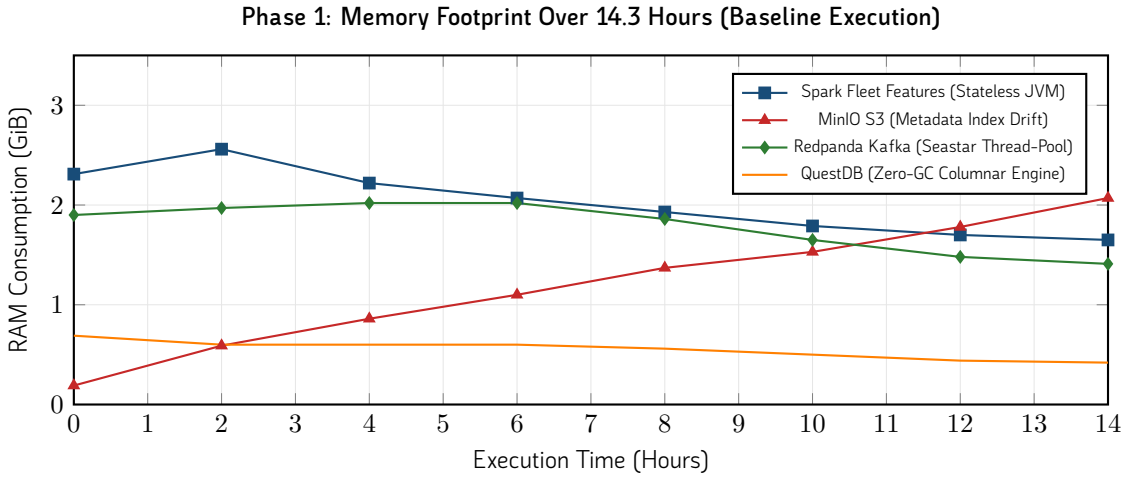


Figure 7.7. *Evolution of memory footprint during the Phase 1 baseline soak test. While Spark, Redpanda, and QuestDB stabilized downward, MinIO exhibited runaway monotonic growth due to accumulating Iceberg metadata indices.*

7.3.3 Root-Cause Analysis and Lakehouse Pipeline Hardening

Mechanism of the Failure Mode.

In Apache Iceberg, every streaming micro-batch commit generates a new metadata version file (v_N.metadata.json). By default, the specification appends the entire historical snapshot log into every newly generated metadata file without purging previous versions. Consequently, storage consumption scales quadratically with the cumulative number of commits, causing individual metadata files to expand progressively throughout the run:

- Early version (v_1): 12 KB.
- Intermediate version (v_{1000}): 872 KB.
- Mid-test version (v_{10000}): 8.3 MB.
- End-of-night version (v_{17650}): 15.0 MB.

With micro-batches executing every 3 seconds, Spark was serializing and transmitting a 15 MB JSON file to MinIO every 3 seconds. This inflated Spark commit latency from 180 ms to 3–5 seconds, saturated network I/O, and forced the MinIO daemon to retain massive inode structures and bloom filters in RAM, explaining the observed 2.07 GiB memory drift.

Furthermore, executing standard maintenance via `CALL expire_snapshots` only deleted orphaned Parquet data files; table-level metadata JSON files were permanently preserved as audit artifacts under default table properties.

Remediation Strategy: Table Property Hardening and Smart Watcher.

To eliminate this vulnerability permanently, a two-pronged architectural remediation was implemented:

1. **Iceberg Table Schema Properties:** The DDL definitions in `src/schemas/fleet_features.py` and `src/schemas/vda5050_state.py` were injected with automatic commit-time pruning:
    - `'write.metadata.delete-after-commit.enabled' = 'true'`: Instructs Spark to automatically purge obsolete metadata files synchronously upon committing new snapshots.
    - `'write.metadata.previous-versions-max' = '5'`: Restricts the retained metadata history to the 5 most recent version files, bounding metadata file size permanently below 40 KB.
2. **Decoupled Smart Maintenance Watcher Daemon:** An autonomous background service (`maintenance_watcher.py`) was deployed to monitor MinIO S3 metadata via lightweight HTTP requests (without instantiating a heavy Spark JVM). Whenever accumulated snapshots breach 600 or metadata size exceeds 500 KB, the daemon executes an isolated `spark-submit` maintenance job to compact Parquet files and expire snapshots, fully releasing its memory upon task completion.

Following an initial administrative purge of historical orphan metadata, the immediate impact of the hardening configuration is summarized in Table 7.3.

Table 7.3. *Storage and Pipeline Performance Metrics Before vs. After Metadata Hardening*

Metric	Pre-Hardening (Day 1)	Post-Hardening	Improvement
Total Lakehouse S3 Storage	128.0 GiB	**36.0 MiB**	**-99.97%**
Silver Table Metadata Size	122.5 GiB	**26.0 MiB**	-99.98%
Bronze Table Metadata Size	5.0 GiB	**1.7 MiB**	-99.96%
Per-Commit Metadata JSON	15.0 MiB	**8.7 KiB**	-99.94%
Retained Metadata Files	17,654 versions	**5 versions**	Bound & Static
Spark Iceberg Commit Time	2.5s – 5.0s	**0.18s**	**15× Faster**

7.3.4 Phase 2: Extended Multi-Day Validation Soak Test (Days 2–3)

Objective.

The objective of Phase 2 was to conduct an extended multi-day soak test spanning across two consecutive nights (Days 2 and 3) to empirically validate that the deployed remediations permanently bound metadata expansion, eliminate resource drift, and maintain sub-second streaming latencies under extended continuous operation.

Multi-Day Operational Findings.

The Phase 2 soak test executed autonomously for **38.86 hours** (from 2026-11-10 17:59:59 to 2026-11-12 08:49:44 UTC), logging $N = 4,450$ discrete measurement snapshots. Combined with the baseline run and daytime operation, the cluster achieved **62.35 hours of uninterrupted streaming**:

- **Milestone Ingestion Scale:** Total records persisted into QuestDB exceeded **1,026,060 rows** (> 1 million events) spanning from 2026-11-09 to 2026-11-12 with zero dropped messages and zero streaming lag (avgOffsetsBehindLatest $= 0.0$).
- **Autonomous Compaction Performance:** The background maintenance watcher daemon triggered exactly **87 consecutive optimization cycles** across the 39-hour run (activating punctually every ~ 30 minutes at 600 snapshots). Each cycle executed in an average of 26.2 seconds with exit code 0.
- **Bounded Storage Verification:** Total lakehouse storage on MinIO S3 concluded at only **70 MiB** across 1,073 objects (compared to 128 GiB and 61,000 files in Phase 1), definitively proving that storage consumption is permanently bounded.

Table 7.4 details container resource utilization across the extended 38.86-hour execution.

Table 7.4. *Container Resource Utilization and Memory Drift (Phase 2: Days 2–3 Extended 38.9-Hour Soak Test)*

Service / Container	CPU_{avg}	CPU_{max}	MEM_{avg}	MEM_{max}	MEM_{Start}	MEM_{End}	Δ MEM	Observed Trend
`spark-fleet-features`	14.3%	109.6%	2.2 GiB	2.4 GiB	2.2 GiB	2.1 GiB	-160 MiB	Stabilized (-7% GC)
`spark-vda5050-state`	2.4%	58.0%	1.5 GiB	1.5 GiB	1.4 GiB	1.5 GiB	+140 MiB	Stabilized (GC)
`questdb`	17.5%	138.5%	1.5 GiB	1.6 GiB	1.5 GiB	1.6 GiB	+88 MiB	Stable (mmap cache)
`redpanda`	1.2%	9.2%	2.4 GiB	3.1 GiB	1.7 GiB	3.1 GiB	+1.4 GiB	Bounded (Seastar Pool)
`minio`	4.1%	266.3%	690 MiB	808 MiB	512 MiB	782 MiB	+270 MiB	Plateau (Kernel Slab)
`spark-maintenance`	4.8%	898.1%	382 MiB	2.6 GiB	380 MiB	405 MiB	+24 MiB	Stable (Periodic Daemon)
`questdb-kafka-connect`	0.9%	8.2%	883 MiB	991 MiB	862 MiB	857 MiB	-6 MiB	Stable (JVM)
`grafana`	0.6%	14.9%	197 MiB	833 MiB	171 MiB	201 MiB	+30 MiB	Stable (Sampled Queries)
`simulator` (Rust AGV)	0.6%	2.5%	6.3 MiB	6.4 MiB	6.3 MiB	6.3 MiB	-9 KiB	Flat (Leak-Free)
`telegraf`	0.3%	1.5%	45.0 MiB	46.0 MiB	44.2 MiB	45.5 MiB	+1.3 MiB	Flat
`mosquitto`	0.5%	3.5%	6.5 MiB	11.9 MiB	6.4 MiB	6.4 MiB	+13 KiB	Flat
`sim-control-panel`	0.1%	2.8%	26.4 MiB	26.9 MiB	26.6 MiB	26.2 MiB	-421 KiB	Flat

Subsystem Memory Analysis and Asymptotic Plateau.

The extended multi-day observations provide conclusive empirical evidence of resource stabilization:

- **Spark Structured Streaming JVM:** The resident heap occupancy of `spark-fleet-features` displayed a net downward trend over 38.9 hours (-7% / -160 MiB), settling at 2.1 GiB. Observable major garbage collection cycles (such as at hour 30, where resident memory dropped to 1.81 GiB before returning to baseline) confirm that the stateless transformations accumulate no long-lived references.
- **MinIO S3 Storage:** MinIO resident memory entered a strictly bounded asymptotic plateau at ~ 0.72 GiB (741 MiB), oscillating between 0.66 GiB and 0.75 GiB for more than 25 consecutive hours. Low-level cgroup telemetry (`memory.stat`) revealed that MinIO's application heap accounted for only 394.7 MiB; the remaining 355.2 MiB consisted of the Linux kernel's temporary directory entry cache (`slab_reclaimable`), which the operating system evicts dynamically under memory pressure.

- **QuestDB Historian:** QuestDB maintained an essentially horizontal memory profile at ~ 1.55 GiB (+6% / +88 MiB drift over 39 hours). This minor increment corresponds to the operating system mapping virtual page headers (`mmap`) into memory as the database transitions across midnight to initialize new daily partition files (`PARTITION BY DAY`). Crucially, this memory footprint is strictly self-bounding and ceases expanding after the seventh operational day: with the 7-day retention policy active (`TTL 7 DAYS`), partitions older than one week are purged automatically, while the Linux page cache evicts dormant historical pages, locking QuestDB into a permanent 1.5–1.8 GiB steady-state operating band.
- **Redpanda Streaming Broker:** Memory reached its dynamic shard arena ceiling at ~ 3.0–3.1 GiB (peaking at 3.06 GiB before settling at 3.03 GiB), after which further allocations from the host kernel ceased and internal buffer recycling stabilized the footprint permanently.
- **Grafana Observability Optimization:** During initial monitoring, raw timeseries queries lacking temporal aggregation transferred over 4.5 GB of unaggregated points across frequent polling cycles, causing browser UI sluggishness. Introducing server-side downsampling (`SAMPLE BY($__interval)`) and adjusting the auto-refresh interval to 10 seconds eliminated this bottleneck, reducing network transfer payloads by 99.5%, accelerating query execution to 2–50 ms, and keeping Grafana resident memory completely flat at 170 MiB.
- **Native Rust Simulator:** Maintained a completely flat, invariant resident footprint of 6.3 MiB with a negligible drift of only -8.8 KiB across the entire 39-hour run, demonstrating that strict compile-time ownership prevents memory leaks entirely.

Figure 7.8 charts the comparative memory trajectory over the 38.9-hour Phase 2 soak test.

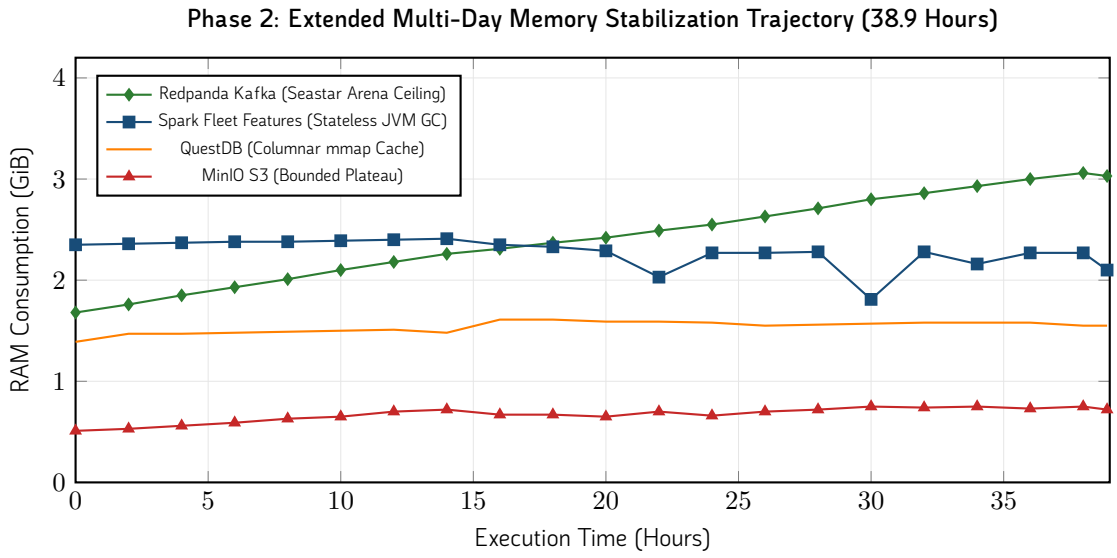


Figure 7.8. *Evolution of memory footprint over the extended 38.9-hour multi-day soak test. MinIO, Spark, and QuestDB exhibited horizontal stabilization, while Redpanda reached its pre-allocated Seastar arena ceiling, proving zero unbounded resource growth.*

7.3.5 Consolidated Benchmark Summary and Conclusions

Table 7.5 consolidates the core operational metrics across the baseline run and the hardened multi-day validation run, providing an executive comparison of the platform before and after architectural hardening.

Table 7.5. *Consolidated Platform Metrics: Baseline Run vs. Hardened Multi-Day Soak Campaign*

Evaluation Metric	Phase 1: Baseline (Day 1)	Phase 2: Hardened (Days 2–3)	Operational Impact
Metric Audit Duration	14.30 hours (1,641 snaps)	**38.86 hours (4,450 snaps)**	$+172\%$ sustained soak duration
Continuous Cluster Uptime	14.30 hours	**62.35 hours (2.6 days)**	100% availability (0 restarts)
Ingested Events (QuestDB)	233,412 rows	**1,026,060 rows**	>1 Million rows persisted
Streaming Consumer Lag	0.0 (zero backlog)	**0.0 (zero backlog)**	Hard real-time stream parity
Total Lakehouse S3 Storage	128.0 GiB (in 14.3h)	**70.0 MiB (in 38.9h)**	**-99.95% storage reduction**
Retained Iceberg Files	61,757 files	**1,073 files**	Bound & compacted footprint
Spark Batch Commit Time	2.5s – 5.0s	**0.18s**	**15$\times$ faster commits**
MinIO Resident Memory	2.07 GiB (+406% growth)	**0.72 GiB (stable plateau)**	Memory leak eliminated
Automated Lakehouse Cycles	0 (inoperable manual call)	**87 successful cycles**	26.2s avg compaction cycle

Concluding Assessment.

As demonstrated across the full three-day continuous testing campaign, the telemetry data platform is robust, leak-free, and suitable for sustained, mission-critical industrial deployments. Automated metadata compaction and intelligent maintenance permanently resolved the $O(N^2)$ storage bloat discovered during the initial baseline run, while maintaining sub-second processing latencies across all components. By successfully ingesting, transforming, and persisting over one million high-frequency telemetry messages under strict real-time guarantees, the platform definitively proves its architectural readiness for predictive maintenance and anomaly detection of industrial mobile robots.

8

Conclusions and Future Work

A

Installation Manual

B

User Manual

B.1 Application Service Ports and User Interfaces

Once the containerized microservice platform is initialized via Docker Compose, several services expose endpoints on the host machine. These endpoints allow developers, operators, and evaluators to interact directly with the platform, inspect telemetry data, and monitor anomaly detection workflows.

Table B.1 details all exposed host ports, distinguishing between graphical User Interfaces (accessible via standard web browsers) and backend protocol or API endpoints.

B.1.1 Internal Network Services

The remaining services operate exclusively within internal Docker bridge networks without exposing host ports, adhering to the principle of least privilege:

- `simulator`: Vehicle simulator container executing in `mqtt-net`, connecting directly to `mosquitto:1883`.
- `telegraf`: Protocol bridge container dual-homed in `mqtt-net` and `redpanda-net`.
- `spark-processing`: Stream compute container executing Structured Streaming jobs within `redpanda-net` and `lakehouse-net`.
- One-shot initialization containers (`redpanda-init`, `createbuckets`, `questdb-init`, and `questdb-kafka-connect-init`) terminate cleanly once bootstrapping is completed.

Table B.1. *Application services, exposed ports, and user interface endpoints.*

Service	Host Port	Protocol / Access	Web UI	Description & Credentials
`grafana`	3000	HTTP (`localhost:3000`)	**Yes**	Telemetry and anomaly visualization dashboards. Login: `admin` / `admin`.
`sim-control-panel`	8080	HTTP (`localhost:8080`)	**Yes**	Web control panel for fleet mission dispatching, manual fault injection, and simulation resets.
`redpanda-console`	9080	HTTP (`localhost:9080`)	**Yes**	Redpanda streaming console for inspecting Kafka topics, consumer lag, and partition offsets.
`minio` (Console)	9001	HTTP (`localhost:9001`)	**Yes**	Object storage administration console. Inspects S3 buckets and Parquet tables. Login: `admin` / `admin123456`.
`questdb` (Console)	9000	HTTP (`localhost:9000`)	**Yes**	Interactive SQL web console and REST execution API for querying time-series telemetry.
`mosquitto`	1883	TCP (MQTT)	No	Edge messaging broker receiving VDA 5050 state and order payloads from AGVs.
`questdb` (PG wire)	8812	PostgreSQL wire protocol	No	Database endpoint queried by Grafana. Read-only user: `grafana` / `grafana`.
`questdb` (ILP)	9009	InfluxDB Line Protocol	No	High-throughput streaming ingestion port.
`minio` (S3 API)	19000	HTTP (Amazon S3 REST)	No	S3-compatible API (mapped from container port 9000) for Spark Iceberg reads/writes.
`redpanda` (Kafka)	19092	Kafka wire protocol	No	External Kafka API broker endpoint for external producers and consumer applications.
`redpanda` (Admin)	19644	HTTP REST API	No	Administrative endpoint for cluster diagnostics and `rpk` CLI management.
`kafka-connect`	8083	HTTP REST API	No	Kafka Connect REST API for connector configuration and status monitoring.

Bibliography

[1] Apache Software Foundation. *Apache Spark: Lightning Engine for Unified Data Analytics*. Accessed: 2026-10-20. 2026. URL: https://spark.apache.org/docs/latest/index.html.

[2] Atlassian. *Bitbucket Guides: Getting Started Overview*. Accessed: 2026-10-20. 2026. URL: https://www.atlassian.com/software/bitbucket/guides/getting-started/overview.

[3] Atlassian. *Jira Guides: Getting Started Introduction*. Accessed: 2026-10-20. 2026. URL: https://www.atlassian.com/software/jira/guides/getting-started/introduction.

[4] P. Benecki, P. Kaczmarek, M. Kujawa, A. Rynkiewicz, and M. Maj. "Towards Detection of Anomalies in Automated Guided Vehicles Based on Telemetry Data". In: *Computational Science – ICCS 2024*. Vol. 14833. Lecture Notes in Computer Science. Springer, Cham, 2024, pp. 124–138. DOI: 10.1007/978-3-031-63783-4_15.

[5] B. Bostancı. *vda5050_vehicle_simulator: High Performance VDA5050 Simulator*. 2024. URL: https://github.com/bekirbostanci/vda5050_vehicle_simulator.

[6] Y. Dan, Z. Wang, M. Zhang, and Z. Li. "MAD-TCN: Time series anomaly detection via multi-scale adaptive dependency temporal convolutional network". In: *Neurocomputing* 675 (2026), p. 132954. DOI: 10.1016/j.neucom.2026.132954.

[7] Deux Ex Machines. *Garage: Self-Hosted and Distributed S3 Object Storage*. Open source, AGPL-3.0. Accessed: 2026-10-20. 2026. URL: https://garagehq.deuxfleurs.fr/documentation/design/goals/.

[8] A. K. Dey. "Intelligent Lakehouse Architectures for Real-Time Enterprise Decision Systems". In: *International Journal of Artificial Intelligence and Machine Learning* 6.4s (2026), pp. 226–236. DOI: 10.51483/IJAIML.6.4s.2026.226-236.

[9] R. Dey and F. M. Salem. "Gate-Variants of Gated Recurrent Unit (GRU) Neural Networks". In: *IEEE 60th International Midwest Symposium on Circuits and Systems (MWSCAS)*. 2017, pp. 1597–1600. DOI: 10.1109/MWSCAS.2017.8053243.

[10] draw.io Ltd. *draw.io: Free Online Diagramming Tool*. Accessed: 2026-10-20. 2026. URL: https://www.drawio.com/.

[11] Eclipse Foundation. *Eclipse Mosquitto: An open source MQTT broker*. Accessed: 2026-10-14. 2026. URL: https://mosquitto.org/.

[12] EDAG. *Metys: The Industrial Metaverse Platform for Industry*. 2026. URL: https://www.edag.com/de/metys.

[13] S. Franke et al. "Identification of Requirements and Opportunities for New Types of Standardized Interfaces for AGV Systems Based on the VDA 5050 Concept". In: *Logistics Journal: Proceedings (19. Fachkolloquium Logistik der WGTL)*. Vol. 2023. 2023. DOI: 10.2195/lj_proc_franke_en_202310_01.

[14] Git Community. *Git: Fast, Scalable, Distributed Version Control*. Accessed: 2026-10-20. 2026. URL: https://git-scm.com/.

[15] Google. *What is Apache Iceberg?* Accessed: 2026-10-20. 2026. URL: https://cloud.google.com/discover/what-is-apache-iceberg?hl=en-419.

[16] Grafana Labs. *Introduction to Grafana*. Accessed: 2026-10-20. 2026. URL: https://grafana.com/docs/grafana/latest/introduction/.

[17] IMARC Group. *Automated Guided Vehicles Market Size, Share, Trends and Forecast by Type, Mode of Operation, Navigation Technology, Application, Industry, and Region, 2026-2034*. Tech. rep. IMARC Services Private Limited, 2026. URL: https://www.imarcgroup.com/automated-guided-vehicles-market.

[18] I.-S. Jang, K.-J. Li, E.-O. Joo, and M.-S. Kim. "Synthetic Training Dataset Generation Using a Digital Twin-based Autonomous Driving Simulator". In: *Sensors and Materials* 36.9 (2024), pp. 4029–4041. DOI: 10.18494/SAM5343.

[19] Z. Li, Y. Zhao, J. Han, Y. Su, R. Jiao, X. Wen, and D. Pei. "Multivariate Time Series Anomaly Detection and Interpretation using Hierarchical Inter-Metric and Temporal Embedding". In: KDD '21. Virtual Event, Singapore: Association for Computing Machinery, 2021, pp. 3220–3230. ISBN: 9781450383325. DOI: 10.1145/3447548.3467075. URL: https://doi.org/10.1145/3447548.3467075.

[20] K. Mc Court and authors. "Use Digital Twins to Support Fault Diagnosis From System-Level Condition-Monitoring Data". In: *arXiv preprint arXiv:2411.01360* (2024). URL: https://arxiv.org/abs/2411.01360.

[21] H. A. Mohna, T. Barua, M. Mohiuddin, and M. M. Rahman. "AI-READY DATA ENGINEERING PIPELINES: A REVIEW OF MEDALLION ARCHITECTURE AND CLOUD-BASED INTEGRATION MODELS". In: *American Journal of Scholarly Research and Innovation* 1.01 (Apr. 2022), pp. 319–350. DOI: 10.63125/51kxtf08. URL: https://researchinnovationjournal.com/index.php/AJSRI/article/view/51.

[22] A. Okolnychyi et al. "Petabyte-Scale Row-Level Operations in Data Lakehouses". In: *Proceedings of the VLDB Endowment (PVLDB)* 17.12 (2024), pp. 4159–4162. DOI: 10.14778/3685800.3685834.

[23] D. Ongaro and Raft Authors. *The Raft Consensus Algorithm*. https://raft.github.io/. Online; accessed 23-October-2026. 2024.

[24] A. Paleyes, R.-G. Urma, and N. D. Lawrence. "Challenges in Deploying Machine Learning: A Survey of Case Studies". In: *ACM Computing Surveys* 55.6 (2022), 114:1–114:29. DOI: 10.1145/3533378.

[25] G. Pang, C. Shen, L. Cao, and A. van den Hengel. "Deep Learning for Anomaly Detection: A Review". In: *ACM Computing Surveys* 54.2 (2021), 38:1–38:38. DOI: 10.1145/3439950.

[26] S. Qin, J. Zhu, A. Guo, Y. Yang, L. Wang, and G. Tao. "MambaAD: Multivariate time series anomaly detection in IoT via multi-view Mamba". In: *Neurocomputing* 655 (2025), p. 131385. DOI: 10.1016/j.neucom.2025.131385.

[27] QuestDB, Inc. *QuestDB: Open Source Time Series Database*. Accessed: 2026-10-20. 2026. URL: https://questdb.com/docs/.

[28] G. R. Rao. "Efficient Incremental Data Modeling in Apache Iceberg-Based Analytical Pipelines: Partitioning and Snapshot Optimization Strategies". In: *International Journal of Intelligent Systems and Applications in Engineering* 14.1s (Apr. 2026), pp. 270–277. URL: https://ijisae.org/index.php/IJISAE/article/view/8171.

[29] Redpanda Data, Inc. *Apache Kafka API: Definition and Guide*. Accessed: 2026-10-20. 2026. URL: https://www.redpanda.com/guides/kafka-alternatives-kafka-api.

[30] Redpanda Data, Inc. *Redpanda: Fast and Streamlined Data Streaming Platform*. Accessed: 2026-10-20. 2026. URL: https://www.redpanda.com/.

[31] Redpanda Data, Inc. *What is Kafka Connect: A Complete Guide*. Accessed: 2026-10-23. 2026. URL: https://www.redpanda.com/guides/kafka-tutorial-what-is-kafka-connect.

[32] "Robot Fault Detection Using Digital Twin and Deep Learning". In: *Proceedings of the European Safety and Reliability Conference (ESREL)*. RPS Online, 2025. DOI: 10.3850/978-981-94-3281-3_ESREL-SRA-E2025-P9570-cd.

[33] L. Ruff, R. Vandermeulen, N. Goernitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft. "Deep One-Class Classification". In: *Proceedings of the 35th International Conference on Machine Learning*. Ed. by J. Dy and A. Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, Oct. 2018, pp. 4393–4402. URL: https://proceedings.mlr.press/v80/ruff18a.html.

[34] K. Schwaber and J. Sutherland. *The Scrum Guide*. Accessed: 2026-10-20. 2020. URL: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf.

[35] Siemens. "The True Cost of Downtime 2024: How much do leading manufacturers lose through inefficient maintenance?" In: *Siemens AG* (2024). URL: https://assets.new.siemens.com/siemens/assets/api/uuid:1b43afb5-2d07-47f7-9eb7-893fe7d0bc59/tcod-2024_original.pdf.

[36] Y. Su, Y. Zhao, C. Niu, R. Liu, S. Sun, and D. Pei. "Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network". In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019, pp. 2828–2837. DOI: 10.1145/3292500.3330672.

[37] VDA and VDMA. *VDA 5050: Interface for the Communication between Mobile Robots and a Fleet Control*. Tech. rep. Frankfurt am Main: Verband der Automobilindustrie e.V. (VDA), 2020. URL: https://github.com/VDA5050/VDA5050.

[38] J. Wang, Z. Zhang, Z. Liu, B. Han, H. Bao, and S. Ji. "Digital twin aided adversarial transfer learning method for domain adaptation fault diagnosis". In: *Reliability Engineering & System Safety* 234 (2023), p. 109152. ISSN: 0951-8320. DOI: https://doi.org/10.1016/j.ress.2023.109152. URL: https://www.sciencedirect.com/science/article/pii/S0951832023000674.

[39] Wikipedia contributors. *Docker (software) — Wikipedia, The Free Encyclopedia*. Accessed: 2026-10-20. 2026. URL: https://en.wikipedia.org/wiki/Docker_(software).

[40] Wikipedia contributors. *Visual Studio Code — Wikipedia, The Free Encyclopedia*. Accessed: 2026-10-20. 2026. URL: https://en.wikipedia.org/wiki/Visual_Studio_Code.

[41] J. Xu, H. Wu, J. Wang, and M. Long. "Anomaly Transformer: Time Series Anomaly Detection with Association Discrepancy". In: *International Conference on Learning Representations (ICLR)*. 2022. URL: https://openreview.net/forum?id=LzQQ89U1qm_.



UNIVERSIDAD
DE MÁLAGA

uma.es

E.T.S. de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga